Coding Question

Create a class **Player** with below attributes:
id – int
matchesPlayed – int
totalRuns – int
name – String
team – String

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class **Solution** with main method:
Implement two static methods – **findAverageTotalRunsOfPlayer** and **findPlayerWithMinimumMatchesPlayed** in Solution class.

findAverageTotalRunsOfPlayer method: Create a static method findAverageTotalRunsOfPlayer in the Solution class. This method will take array of Player objects and returns the average TotalRuns of Player if found else return 0 if not found.

findPlayerWithMinimumMatchesPlayed method: Create a static method findPlayerWithMinimumMatchesPlayed in the Solution class. This method will take array of Player objects and returns the Player object having the maximum MatchesPlayed if found else return null if not found.

These methods should be called from the main method.

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

```
import java.util.Scanner;

class Player

{

   int id;

   int matchesPlay;

   int totalRun;

   String name;

   String team;

   public Player(int id, int matchesPlay, int totalRun, String name, String team) {

      super();

      this.id = id;

      this.matchesPlay = matchesPlay;

      this.totalRun = totalRun;

      this.name = name;

      this.team = team;

}

public int getId() {

   return id;

}

public void setId(int id) {

   this.id = id;

}

public String getName() {

   return name;

}
```

```
public void setName(String name) {
   this.name = name;
}
public String getTeam() {
   return team;
}
public void setTeam(String team) {
   this.team = team;
}
public int getMatchPlay() {
   return matchesPlay;
}
public void setMatchPlay(int matchesPlay) {
      this.matchesPlay = matchesPlay;
   }
   public int getTotalRun() {
      return totalRun;
   }
   public void setTotalRun(int totalRun) {
      this.totalRun = totalRun;
   }
}
public class Solution1
{
   public static void main(String[] args)
```

```
{
   Scanner in = new Scanner(System.in);
   int n=in.nextInt();
   Player[] p = new Player[n];
   for(int i=0;i<p.length;i++)
   {
      int id = in.nextInt();
      int matchesPlay = in.nextInt();
      int totalRun = in.nextInt();in.nextLine();
      String name = in.nextLine();
      String team = in.nextLine();
      p[i] = new Player(id, matchesPlay, totalRun, name, team);
   }
double r = findAverageTotalRunsOfPlayer(p);
if(r == 0){System.out.println("No Player found with mentioned attribute.");}
else{
   System.out.println("Average of totalRuns "+r);
}
Player pr = findPlayerWithMaximumMatchesPlayed(p);
if(pr == null){System.out.println("No Player found with mentioned attribute.");}
else{
   System.out.println("id-"+pr.getId());
   System.out.println("matchesPlayed-"+pr.getMatchPlay());
   System.out.println("totalRuns-"+pr.getTotalRun());
   System.out.println("name-"+pr.getName());
```

```
        System.out.println("team-"+pr.getTeam());
    }
    in.close();
}
public static double findAverageTotalRunsOfPlayer(Player[] p){
    double sum = 0;
    for(int i=0; i< p.length; i++){
        sum = sum + p[i].getTotalRun();
    }
    if(sum == 0)
        return 0;
    else
        return (sum/4);
}
public static Player findPlayerWithMaximumMatchesPlayed(Player[] player)
{
    //method logic
    Player maxPlayed=player[0];
    int maxMatches=0;
    for(int i=0; i<player.length;i++){
        if (player[i].getMatchPlay()>maxMatches){
            maxPlayed=player[i];
            maxMatches=player[i].getMatchPlay();
        }
    }
```

```
        if(maxMatches==0){return null;}
        else{
          return maxPlayed;
        }
    }
}
```

Coding Question

Shopping Items

Create a class **ShoppingItem** with the below attributes:
itemId – int
itemType – String
itemName – String
itemprice – double
yearOfMfg – int

The above attributes should be private, write getters, setters and parameterized constructor as request.

Create class Solution with main method.
Implement two static methods – **getItemCountForYear** and **getHighestPricedItem** in Solution class.

getItemCountForYear method:
This method will take two input parameters – array of ShoppingItem objects and and integer representing year.
The method will return the total number of items from array of ShoppingItem objects, it the year in yearOfMfg attribute of the ShoppingItem object matches the integer parameter passed.
If no item with given yearOfMfg is present in the array, then the method should return 0.

```
import java.util.*;


public class Solution {
```

```
public static void main(String[] args) {
		// TODO Auto-generated method stub
		int itemId;
		String itemType;
		String itemName;
		double itemPrice;
		int yearofMFG;
		ShoppingItem[] shop = new ShoppingItem[4];
		Scanner sc = new Scanner(System.in);
		for(int i=0;i<shop.length;i++) {
			itemId = sc.nextInt();
			sc.nextLine();
			itemType = sc.nextLine();
			itemName = sc.nextLine();
			itemPrice = sc.nextDouble();
			yearofMFG = sc.nextInt();
			shop[i] = new ShoppingItem(itemId, itemType, itemName, itemPrice,
yearofMFG);
			}
		int yearofMFG1 = sc.nextInt();
		sc.nextLine();
		String itemType1 = sc.nextLine();
		int count = getItemCountForYear(shop,yearofMFG1);
		if(count>0) {
			System.out.println(count);
		}
```

```
        else {
                System.out.println("no Item");
        }

        ShoppingItem shop1 = getHighestPricedItem(shop,itemType1);
        if(shop1 == null) {
                System.out.println("no item");
        }
        else {

System.out.println(shop1.getItemName()+";"+shop1.getItemPrice()+";"+shop1.getItemType());
        }
}
public static int getItemCountForYear(ShoppingItem[] shop,int yearofMFG) {
        int c =0;
        for(int i=0;i<shop.length;i++) {
                if(shop[i].getYearofMFG() == yearofMFG ) {
                        c++;
                }
        }
        return c;
}
public static ShoppingItem getHighestPricedItem(ShoppingItem[] shop,String itemType) {
        int j =0;
        for(int i =0;i<shop.length;i++) {
                if(shop[i].getItemType().equalsIgnoreCase(itemType)) {
```

```
                    j++;
            }
}
if(j==0)
            return null;
ShoppingItem[] shop2 = new ShoppingItem[j];
j=0;
for(int i =0;i<shop.length;i++) {
            if(shop[i].getItemType().equalsIgnoreCase(itemType)) {
                        shop2[j++] = shop[i];
            }
}
for(int i=0;i<shop2.length;i++) {
                        for(int k=i+1;k<shop2.length;k++) {
                                    if(shop2[i].getItemPrice()<shop2[k].getItemPrice()) {
                                                ShoppingItem temp = shop2[i];
                                                shop2[i] = shop2[k];
                                                shop2[k] = temp;
                                    }
                        }
                }
                        return shop2[0];
        }
}
class ShoppingItem{
```

```
	int itemId;
	String itemType;
	String itemName;
	double itemPrice;
	int yearofMFG;
	public ShoppingItem(int itemId, String itemType, String itemName, double itemPrice, int yearofMFG) {
		super();
		this.itemId = itemId;
		this.itemType = itemType;
		this.itemName = itemName;
		this.itemPrice = itemPrice;
		this.yearofMFG = yearofMFG;
	}
public int getItemId() {
		return itemId;
}

public String getItemType() {
		return itemType;
}

public String getItemName() {
		return itemName;
}
```

```
        public double getItemPrice() {
                return itemPrice;
        }

        public int getYearofMFG() {
                return yearofMFG;
        }
}
```

Coding Question

Create a class **Employee** with the below attribute:
employeeId – int
employeeName – String
salary – double
grade – String
location – String
yearsOfExperience – int

Write getters, setters for the above attributes. Create constructor which takes parameter in the above sequence.

Create class Solution with main method.
Implement two static methods – **findCountOfEmployeesByLoc** and **findHighestSalaryEmpDetailsWithGivenGrade** in Solution class.

findCountOfEmployeesByLoc method: This method will take two input parameters – array of Employees objects and string parameter location. The method will return the count of employees from Employee objects working in the given location if the yearsOfExperience is greater than 2. If no employee with the given location ad with the mentioned yearsOfExperience is present in the array of Employee objects, then the method should return 0.

findHighestSalaryEmpDetailsWithGivenGrade method: This method will take two input parameters – array of Employee objects and String parameter grade. This method will return

---

```
import java.util.*;
```

```
class Employee{
   int employeeId;
   String employeeName;
   double salary;
   String grade;
   String location;
   int yearsOfExperience;
   Employee(int employeeId, String employeeName, double salary, String grade, String location, int yearsOfExperience){
      this.employeeId = employeeId;
      this.employeeName = employeeName;
      this.salary = salary;
      this.grade = grade;
      this.location = location;
      this.yearsOfExperience = yearsOfExperience;
}
int getEmployeeId(){
   return employeeId;
}
String getEmployeeName(){
   return employeeName;
}
double getSalary(){
   return salary;
}
String getGrade(){
```

```
        return grade;
    }
    String getLocation(){
        return location;
    }
    int getYearsOfExperience(){
        return yearsOfExperience;
    }
}

public class Solution{
    public static void main(String[] args){
int employeeId;
String employeeName;
double salary;
String grade;
String location;
int yearsOfExperience;
Employee[] emp = new Employee[4];
Scanner s = new Scanner(System.in);
for(int i=0;i<emp.length;i++){
    employeeId = s.nextInt();s.nextLine();
    employeeName = s.nextLine();
    salary = s.nextDouble();s.nextLine();
    grade = s.nextLine();
```

```
    location = s.nextLine();
    yearsOfExperience = s.nextInt();
    emp[i] = new Employee(employeeId, employeeName, salary, grade, location, yearsOfExperience);
}
s.nextLine();
String newLocation = s.nextLine();
String newGrade = s.nextLine();
s.close();
int countOfEmployees = findCountOfEmployeesByLoc(emp, newLocation);
Employee emp1 = findHighestSalaryEmpDetailsWithGivenGrade(emp, newGrade);

if(countOfEmployees == 0)
        System.out.println("There are no employees present in the given location and
yearsOfExperience");
      else
        System.out.println(countOfEmployees);

      if(emp1 == null)
        System.out.println("No employees present in the given grade");
      else

System.out.println(emp1.getEmployeeId()+":"+emp1.getEmployeeName()+":"+emp1.getLocation());
  }
  public static int findCountOfEmployeesByLoc(Employee[] emp, String location){
    int count = 0;
    for(int i=0;i<emp.length;i++){
```

```
        if(location.equalsIgnoreCase(emp[i].getLocation())){
            if(emp[i].getYearsOfExperience()>2){
                count++;
            }
        }
    }
    return count;
}
public static Employee findHighestSalaryEmpDetailsWithGivenGrade(Employee[] emp, String grade){
    int count = 0;
    for(int i=0;i<emp.length;i++){
        if(grade.equalsIgnoreCase(emp[i].getGrade()))
            count++;
    }
    if(count == 0) return null;
    Employee[] emp1 = new Employee[count];
    count = 0;
    for(int i=0;i<emp.length;i++){
        if(grade.equalsIgnoreCase(emp[i].getGrade()))
            emp1[count++] = emp[i];
    }
    for(int i=0;i<emp1.length;i++){
        for(int j=i+1;j<emp1.length;j++){
            if(emp1[i].getSalary()>emp1[j].getSalary()){
                Employee temp = emp1[i];
```

```
                emp1[i] = emp1[j];
                emp1[j] = temp;
            }
        }
    }
    return emp1[emp1.length-1];
  }
}
```

Create a class **Employee** with below attributes:
empId – int
empName – String
mgrId – int
deptName – String
salary – double

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class **Solution** with main method.
Implement two static methods – **empsWithSalMoreThanAvg** and **getManagerWithMaxSalary** in Solution class.

empsWithSalMoreThanAvg method: This method will take one input parameter – array of Employee objects. The method will return the number of Employee objects whose salary is greater than the average salary. If no employee object contains salary more than average salary, then the method will return 0.

getManagerWithMaxSalary method: This method will take one input parameter – array of Employee objects. The method will return the Employee object, who is getting the maximum salary of all employees if the mgrId is not 0.
If there are no employees with mgrId as non zero, then the method should return null.

Note: All Employee object would have the salary greater than 0.

```
import java.util.Scanner;


class Employee{
   private int empId;
   private String empName;
   private int mgrId;
   private String deptName;
   private double salary;
   public Employee(int empId, String empName, int mgrId, String deptName, double salary){
      this.empId = empId;
      this.empName = empName;
      this.mgrId = mgrId;
      this.deptName = deptName;
      this.salary = salary;
   }
public int getEmpId(){
   return empId;
}
public String getEmpName(){
   return empName;
}
public int getMgrId(){
   return mgrId;
}
public double getSalary(){
```

```
        return salary;
    }
}

public class Solution{
    public static void main(String[] args){
        int empId;
        String empName;
        int mgrId;
        String deptName;
        double salary;
        Scanner s = new Scanner(System.in);
        Employee[] emp = new Employee[5];
        for(int i=0;i<emp.length;i++){
            empId = s.nextInt();s.nextLine();
            empName = s.nextLine();
            mgrId = s.nextInt();s.nextLine();
            deptName = s.nextLine();
            salary = s.nextDouble();
            emp[i] = new Employee(empId, empName, mgrId, deptName, salary);
        }

        int count = empsWithSalMoreThanAvg(emp);
        Employee employee = getManagerWithMaxSalary(emp);
```

```
    if(count > 0)
       System.out.println(count);
    else
       System.out.println("No such Employees");

    if(employee == null)
       System.out.println("No Managers");
    else
       System.out.println(employee.getEmpId()+"#"+employee.getEmpName());
}
public static int empsWithSalMoreThanAvg(Employee[] emp){
    double avgSalary = 0, sum=0;
int count = 0;
for(int i=0;i<emp.length;i++)
   if(emp[i].getSalary() > 0) {
       sum += emp[i].getSalary();
       count++;
   }
avgSalary = sum / count;
count = 0;
for(int i=0;i<emp.length;i++){
   if(emp[i].getSalary() > avgSalary)
       count++;
}
return count;
```

```
}
public static Employee getManagerWithMaxSalary(Employee[] emp){
   int count = 0;
   for(int i=0;i<emp.length;i++)
      if(emp[i].getMgrId() != 0)
         count++;
   if(count == 0) return null;
   double maxSalary = -999;
   for(int i=0;i<emp.length;i++){
      if(emp[i].getMgrId() != 0)
         if(emp[i].getSalary() > maxSalary)
            maxSalary = emp[i].getSalary();
      }
      for(int i=0;i<emp.length;i++){
         if(emp[i].getSalary() == maxSalary)
            return emp[i];
      }
      return null;
   }
}
```

Coding Question

Create a class **TollPlaza** with below attributes:
vehicleId – String
vehicleType – String
modeOfPayment – String
fare – double

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class Solution with main method.
Implement two static methods – **getPaymentModeWiseTotalCollection** and **findVehicleWithVehicleId** in Solution class.

getPaymentModeWiseTotalCollection method: This method will take two input parameters – array of TollPlaza objects and String parameter modeOfPayment. This method will return total fare collection of TollPlaza, which means it should sum up the fare attribute of TollPlaza objects if modeOfPayment is matched. If no TollPlaza objects with the given modeOfPayment is present in the array of TollPlaza objects, then the method should return 0.

findVehicleWithVehicleId method – This method will take two input parameters – array of TollPlaza objects and String parameter vehicleId. The method will return the TollPlaza object. If the input String parameter matches with the vehicleId attribute of the TollPlaza object. If any of the conditions are not met, then the method should return null.

Coding Question

Create a class **TollPlaza** with below attributes:
vehicleId – String
vehicleType – String
modeOfPayment – String
fare – double

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class Solution with main method.
Implement two static methods – **getPaymentModeWiseTotalCollection** and **findVehicleWithVehicleId** in Solution class.

getPaymentModeWiseTotalCollection method: This method will take two input parameters – array of TollPlaza objects and String parameter modeOfPayment. This method will return total fare collection of TollPlaza, which means it should sum up the fare attribute of TollPlaza objects if modeOfPayment is matched. If no TollPlaza objects with the given modeOfPayment is present in the array of TollPlaza objects, then the method should return 0.

findVehicleWithVehicleId method – This method will take two input parameters – array of TollPlaza objects and String parameter vehicleId. The method will return the TollPlaza object. If the input String parameter matches with the vehicleId attribute of the TollPlaza object. If any of the conditions are not met, then the method should return null.

Coding Question

Create a class **TollPlaza** with below attributes:
vehicleId – String
vehicleType – String
modeOfPayment – String
fare – double

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class Solution with main method.
Implement two static methods – **getPaymentModeWiseTotalCollection** and **findVehicleWithVehicleId** in Solution class.

getPaymentModeWiseTotalCollection method: This method will take two input parameters – array of TollPlaza objects and String parameter modeOfPayment. This method will return total fare collection of TollPlaza, which means it should sum up the fare attribute of TollPlaza objects if modeOfPayment is matched. If no TollPlaza objects with the given modeOfPayment is present in the array of TollPlaza objects, then the method should return 0.

findVehicleWithVehicleId method – This method will take two input parameters – array of TollPlaza objects and String parameter vehicleId. The method will return the TollPlaza object. If the input String parameter matches with the vehicleId attribute of the TollPlaza object. If any of the conditions are not met, then the method should return null.

Coding Question

Create a class **TollPlaza** with below attributes:
vehicleId – String
vehicleType – String
modeOfPayment – String
fare – double

Write getters, setters and parameterized constructor in the above mentioned attribute sequence as required.

Create class Solution with main method.
Implement two static methods – **getPaymentModeWiseTotalCollection** and **findVehicleWithVehicleId** in Solution class.

getPaymentModeWiseTotalCollection method: This method will take two input parameters – array of TollPlaza objects and String parameter modeOfPayment. This method will return total fare collection of TollPlaza, which means it should sum up the fare attribute of TollPlaza objects if modeOfPayment is matched. If no TollPlaza objects with the given modeOfPayment is present in the array of TollPlaza objects, then the method should return 0.

findVehicleWithVehicleId method – This method will take two input parameters – array of TollPlaza objects and String parameter vehicleId. The method will return the TollPlaza object. If the input String parameter matches with the vehicleId attribute of the TollPlaza object. If any of the conditions are not met, then the method should return null.

```
import java.util.Scanner;


public class Solution {

  public static void main(String[] args){

     String vehicleId;

     String vehicleType;

     String modeOfPayment;

     double fare;

     TollPlaza[] tollplazaArr = new TollPlaza[5];

     Scanner s = new Scanner(System.in);
```

```
for(int i=0;i<tollplazaArr.length;i++) {
    vehicleId = s.nextLine();
    vehicleType = s.nextLine();
    modeOfPayment = s.nextLine();
    fare = s.nextDouble();
    s.nextLine();
    tollplazaArr[i] = new TollPlaza(vehicleId, vehicleType, modeOfPayment, fare);
}
String newModeOfPayment = s.nextLine();
String newVehicleId = s.nextLine();
s.close();
    double totalCollection = getPaymentModeWiseTotalCollection(tollplazaArr, newModeOfPayment);
    TollPlaza findVehicle = findVehicleWithVehicleId(tollplazaArr, newVehicleId);

    if(totalCollection == 0)
      System.out.println("The given mode of payment is not available");
    else
      System.out.println(totalCollection);

    if(findVehicle == null)
      System.out.println("The given vehicle ID is incorrect");
    else
System.out.println(findVehicle.getVehicleId()+"#"+findVehicle.getVehicleType()+"#"+findVehicle.getMo
deOfPayment()+"#"+findVehicle.getFare());
```

```
  }
  public static double getPaymentModeWiseTotalCollection(TollPlaza[] tollplazaArr, String
modeOfPayment){
    double totolFare = 0;
    for(int i=0;i<tollplazaArr.length;i++){
      if(modeOfPayment.equalsIgnoreCase(tollplazaArr[i].getModeOfPayment()))
        totolFare += tollplazaArr[i].getFare();
    }
    return totolFare;
  }
  public static TollPlaza findVehicleWithVehicleId(TollPlaza[] tollplazaArr, String vehicleId){
    for(int i=0;i<tollplazaArr.length;i++){
      if(vehicleId.equalsIgnoreCase(tollplazaArr[i].getVehicleId()))
        return tollplazaArr[i];
      }
      return null;
    }
}

class TollPlaza{
  String vehicleId;
  String vehicleType;
  String modeOfPayment;
  double fare;
  TollPlaza(String vehicleId, String vehicleType, String modeOfPayment, double fare){
    this.vehicleId = vehicleId;
```

```
    this.vehicleType = vehicleType;
    this.modeOfPayment = modeOfPayment;
    this.fare = fare;
}
String getVehicleId(){
    return vehicleId;
}
String getVehicleType(){
    return vehicleType;
}
String getModeOfPayment(){
    return modeOfPayment;
    }
    double getFare(){
        return fare;
    }
}
```

Policy Management

Create a class **Policy** with below attributes:
policyId – int
policyName – String
paymentMode – String
policyAmount – double
claimed – boolean

Write getters, setters for the above attributes.

Create constructor which takes parameter in the above sequence except claimed attribute.

Create class Solution with main method.
Implement two static methods – **getPolicyAmount** and **getPolicyWithInAmountRange** in Solution class.

getPolicyAmount method:
This method will take four input parameters – array of Policy objects, int parameter policyId, boolean parameter claimed, int parameter percentageAlreadyClaimed if the input parameter policyId matches with the policyId attribute of the Policy object and input parameter claimed is true then the input parameter percentageAlreadyClaimed is deducted from the policyAmount.

```
import java.util.*;


public class Solution
{
        public static void main (String[] args)
        {
                // your code goes here
                Scanner sc=new Scanner(System.in);
                Policy[] policy = new Policy[4];
                for(int i=0;i<policy.length;i++)
                {
```

```
            int policyId=sc.nextInt();sc.nextLine();
            String policyName=sc.nextLine();
            String paymentMode=sc.nextLine();
            double policyAmount=sc.nextDouble();
            policy[i]=new Policy(policyId, policyName, paymentMode, policyAmount);
}
int newPolicyId=sc.nextInt();
int percentageAlreadyClaimed=sc.nextInt();
boolean claimed=sc.nextBoolean();
double minAmount=sc.nextDouble();
double maxAmount=sc.nextDouble();
sc.close();
            double newPolicyAmount = getPolicyAmount(policy, newPolicyId, claimed,
percentageAlreadyClaimed);
            if(newPolicyAmount==0){
                  System.out.println("Policy is not claimed");}
            else
                  System.out.println("Policy Amount after claim is :"+newPolicyAmount);

            Policy newPolicy = getPolicyWithInAmountRange(policy, minAmount, maxAmount);
            if(newPolicy!=null){
                  System.out.print("Policy Id:"+newPolicy.getPolicyId());
                  System.out.print(" Policy Name:"+newPolicy.getPolicyName());
                  System.out.print(" Total Amount:"+newPolicy.getPolicyAmount());}
            else
```

```
                                                        within the specified range");
        }
        public static double getPolicyAmount(Policy[] policy,int newPolicyId, boolean claimed,int
percentageAlreadyClaimed)
        {
                double newPolicyAmount=0;
                for(int i=0;i<policy.length;i++)
                {
                        if(policy[i].getPolicyId()==newPolicyId && claimed==true)
                        {
                                newPolicyAmount = policy[i].getPolicyAmount() -
((policy[i].getPolicyAmount()*percentageAlreadyClaimed)/100);
                                policy[i].setPolicyAmount(newPolicyAmount);
                        }
                }
                return newPolicyAmount;

        }
        public static Policy getPolicyWithInAmountRange(Policy[] policy,double minAmount,double
maxAmount)
        {
          for(int i=0;i<policy.length;i++) {
                if(policy[i].getPolicyAmount() > minAmount &&
policy[i].getPolicyAmount()<maxAmount)
                        return policy[i];
          }
          return null;
```

```
        }
}
class Policy
{
        int policyId;
        String policyName;
        String paymentMode;
        double policyAmount;
        Policy(int policyId,String policyName,String paymentMode,double policyAmount)
        {
                this.policyId = policyId;
                this.policyName = policyName;
                this.paymentMode = paymentMode;
                this.policyAmount = policyAmount;
}
int getPolicyId() {
        return policyId;
}
String getPolicyName() {
        return policyName;
}
String getPaymentMode() {
        return paymentMode;
}
double getPolicyAmount() {
```

```
            return policyAmount;
        }
        void setPolicyAmount(double policyAmount) {
            this.policyAmount = policyAmount;
        }
}
```

Q1 :-

```
import java.util.*;


public class Solution {
	public static void main(String[] args) {
		int billId;
		String custName;
		int units;
		Scanner s = new Scanner(System.in);
		Bill[] bill = new Bill[5];
		for(int i=0;i<bill.length;i++) {
			billId = s.nextInt();s.nextLine();
			custName = s.nextLine();
			units = s.nextInt();
			bill[i] = new Bill(billId, custName, units);
}
double minBillAmnt = s.nextDouble();
double maxBillAmnt = s.nextDouble();
s.close();
int noOfBills = findBillsInTheAmntRange(bill, minBillAmnt, maxBillAmnt);
Bill b = getBillWithHighAmnt(bill);


if(noOfBills == 0)
		System.out.println("No bills with given amount");
else
```

```
                System.out.println(noOfBills);


        if(b == null)
                System.out.println("No bill object found");
        else
                System.out.println(b.getBillId()+" "+b.getCustName()+" "+b.getUnits());
}
public static int findBillsInTheAmntRange(Bill[] bills, double minAmnt, double maxAmnt) {
        double billAmnt = 0;int count = 0;
        for(int i=0;i<bills.length;i++) {
                if(bills[i].getUnits()>=1000)
                        billAmnt = bills[i].getUnits() * 5;
                else if(bills[i].getUnits()>=500 && bills[i].getUnits()<1000)
                        billAmnt = bills[i].getUnits() * 3;
                else if(bills[i].getUnits()<500)
                        billAmnt = bills[i].getUnits() * 1;
                if(billAmnt >= minAmnt && billAmnt <= maxAmnt) count++;
        }
        return count;
}
public static Bill getBillWithHighAmnt(Bill[] bills) {
        int maxUnits=-999;
        for(int i=0;i<bills.length;i++) {
                if(bills[i].getUnits()>=maxUnits)
                        maxUnits = bills[i].getUnits();
```

```
                }
                for(int i=0;i<bills.length;i++) {
                        if(maxUnits == bills[i].getUnits())
                                return bills[i];
                }
                return null;
        }
}
class Bill{
        int billId;
        String custName;
        int units;
        Bill(int billId, String custName, int units){
                this.billId = billId;
                this.custName = custName;
                this.units = units;
        }
        int getBillId() {
                return billId;
        }
        String getCustName() {
                return custName;
        }
        int getUnits() {
                return units;
```

```
            }
}
Q2-
import java.util.*;
public class Solution{

    public static void main(String []args){
      Scanner s = new Scanner(System.in);
      Bus[] bus = new Bus[4];
      String busNo, fromStation, toStation;
      double ticketCost;
      for(int i=0;i<bus.length;i++){
        busNo = s.nextLine();
        fromStation = s.nextLine();
        toStation = s.nextLine();
        ticketCost = s.nextDouble();s.nextLine();
        bus[i] = new Bus(busNo, fromStation, toStation, ticketCost);
    }
String getFromStation = s.nextLine();
String getToStation = s.nextLine();
s.close();
Bus b = getBus(bus, getFromStation, getToStation);
double cost = getSumOfMinMaxTicketCost(bus);
 if(cost>0)
    System.out.println(cost);
```

```
    else
        System.out.println("Travel is free of cost");

    if(b==null)
        System.out.println("No Bus found");
    else
        System.out.println(b.getBusNo());

}
public static Bus getBus(Bus[] bus, String fromStation, String toStation){
    for(int i=0;i<bus.length;i++){
        if(fromStation.equalsIgnoreCase(bus[i].getFromStation()) &&
toStation.equalsIgnoreCase(bus[i].getToStation()))
            return bus[i];
        }
        return null;
    }
    public static double getSumOfMinMaxTicketCost(Bus[] bus){
        double minCost = 9999999, maxCost = -9999999;
        for(int i=0;i<bus.length;i++){
            if(bus[i].getTicketCost() <= minCost)
                minCost = bus[i].getTicketCost();
            if(bus[i].getTicketCost() >= maxCost)
                maxCost = bus[i].getTicketCost();
        }
        return minCost+maxCost;
```

```
    }
}
class Bus{
   String busNo, fromStation, toStation;
   double ticketCost;
   Bus(String busNo, String fromStation, String toStation, double ticketCost){
      this.busNo = busNo;
      this.fromStation = fromStation;
      this.toStation = toStation;
      this.ticketCost = ticketCost;
   }
   String getBusNo(){
      return busNo;
   }
   String getFromStation(){
      return fromStation;
   }
   String getToStation(){
      return toStation;
   }
   double getTicketCost(){
      return ticketCost;
   }
}
```

Q3-

```
import java.util.*;

public class Solution{

 public static void main(String[] args) {

  Scanner s = new Scanner(System.in);

  int studentId, sub1, sub2, sub3;

  String studentName;

  Student[] student = new Student[4];

  for(int i=0;i<student.length;i++) {

   studentId = s.nextInt();s.nextLine();

   studentName = s.nextLine();

   sub1 = s.nextInt();

   sub2 = s.nextInt();

   sub3 = s.nextInt();

   student[i] = new Student(studentId, studentName, sub1, sub2, sub3);

}

s.close();

int passedStudents = findPassedStudentCount(student);

Student stu = getTopStudent(student);


if(passedStudents == 0)

 System.out.println("No Students Passed");

else

 System.o


if(stu == null)
```

```
   System.out.println("No one has passed");
  else
   System.out.println(stu.getStudentId()+"#"+stu.getStudentName());
}
public static int findPassedStudentCount(Student[] students) {
 int count = 0;
 for(int i=0;i<students.length;i++) {
  if(students[i].getSub1() >= 60 && students[i].getSub2() >= 60 && students[i].getSub3()>=60)
    count++;
 }
 return count;
}
public static Student getTopStudent(Student[] students) {
 int count = 0;
 for(int i=0;i<students.length;i++) {
  if(students[i].getSub1() >= 60 && students[i].getSub2() >= 60 && students[i].getSub3()>=60)
    count++;
 }
 if(count == 0) return null;
 Student[] s = new Student[count];
 count = 0;
 for(int i=0;i<students.length;i++) {
  if(students[i].getSub1() >= 60 && students[i].getSub2() >= 60 && students[i].getSub3()>=60)
    s[count++] = students[i];
 }
```

```
    int sum = s[0].getSub1()+s[0].getSub2()+s[0].getSub3();
    for(int i=1;i<s.length;i++) {
      if(s[i].getSub1()+s[i].getSub2()+s[i].getSub3() >= sum)
        sum = s[i].getSub1()+s[i].getSub2()+s[i].getSub3();
    }
    for(int i=0;i<s.length;i++) {
      if(sum == s[i].getSub1()+s[i].getSub2()+s[i].getSub3())
        return s[i];
    }
    return null;
  }
}
class Student{
 int studentId, sub1, sub2, sub3;
 String studentName;
 Student(int studentId, String studentName, int sub1, int sub2, int sub3){
   this.studentId = studentId;
   this.studentName = studentName;
   this.sub1 = sub1;
   this.sub2 = sub2;
   this.sub3 = sub3;
 }
 int getStudentId() {
   return studentId;
 }
```

```
String getStudentName() {
 return studentName;
}
int getSub1() {
 return sub1;
}
int getSub2() {
 return sub2;
}
int getSub3() {
 return sub3;
}
```

Q-4

```
import java.util.*;
public class Solution{
   public static void main(String[] args){
      int empId;
      String empName;
      int mgrId;
      String deptName;
      double salary;
      Scanner s = new Scanner(System.in);
      Employee[] emp = new Employee[4];
      for(int i=0;i<emp.length;i++){
```

```
    empId = s.nextInt();
    empName = s.nextLine();
    mgrId = s.nextInt();s.nextLine();
    deptName = s.nextLine();
    salary = s.nextDouble();
    emp[i] = new Employee(empId, empName, mgrId, deptName, salary);
}
s.nextLine();
double r1=s.nextDouble();
double r2=s.nextDouble();

int res=findempsWithSalRange(emp,r1,r2);
    if(res==0)
      System.out.println("No Employee in the given range");
    else
      System.out.println(res);

    Employee res2=getEmployeeWitMaxSal(emp);
    if(res2==null)
      System.out.println("NOT FOUND");
    else
      System.out.println(res2.empId+"#"+res2.empName+"#"+res2.salary);
}
public static int findempsWithSalRange(Employee[] emp,double r1,double r2){
int c=0;
```

```
for(int i=0;i<emp.length;i++)
{
   if(emp[i].getSalary()>r1 && emp[i].getSalary()<r2){
      c++;
  }
 }
 if (c==0)
  return 0;
 else
 return c;
}
public static Employee getEmployeeWitMaxSal(Employee[] emp)
{
 for(int i = 0; i<emp.length-1; i++)
 {
 for (int j = i+1; j<emp.length; j++)
 {
  if(emp[i].getSalary() < emp[j].getSalary())
  {
   Employee t=emp[i];
   emp[i]=emp[j];
   emp[j]=t;
  }

 }
```

```
    }
    return emp[0];


    }
  }
class Employee
{
 int empId;
 String empName;
 int mgId;
 String deptName;
 double salary;
public int getEmpId() {
 return empId;
}
public Employee(int empId, String empName, int mgId, String deptName, double salary) {
 super();
 this.empId = empId;
 this.empName = empName;
 this.mgId = mgId;
 this.deptName = deptName;
 this.salary = salary;
}
public void setEmpId(int empId) {
 this.empId = empId;
```

```
}
public String getEmpName() {
 return empName;
}
public void setEmpName(String empName) {
 this.empName = empName;
}
public int getMgId() {
 return mgId;
}
public void setMgId(int mgId) {
 this.mgId = mgId;
}
public String getDeptName() {
 return deptName;
}
public void setDeptName(String deptName) {
 this.deptName = deptName;
}
public double getSalary() {
 return salary;
}
public void setSalary(double salary) {
 this.salary = salary;
}
```

```
}
```

Q5-

```
import java.util.*;
import java.lang.*;
public class Solution1{
   public static void main(String[] args){
      int contestId;
      String contestName;
      String contestWinner;
      int points;
      String category;
      Scanner s = new Scanner(System.in);
      Contest[] contests = new Contest[4];
      for(int i=0; i<contests.length; i++){
         contestId = s.nextInt();s.nextLine();
         contestName = s.nextLine();
         contestWinner = s.nextLine();
         points = s.nextInt();s.nextLine();
         category = s.nextLine();
         contests[i] = new Contest(contestId, contestName, contestWinner, points, category);
      }
      String newContestWinner = s.nextLine();
      String newCategory = s.nextLine();
      s.close();
      int totalPoints = findTotalPointsBasedOnWinner(contests, newContestWinner);
```

```
    if(totalPoints == 0)
        System.out.println("Contest Winner not found");
    else
        System.out.println(totalPoints);

    Contest contest = getContestWithSecondHighestPoints(contests, newCategory);
    if(contest == null)
        System.out.println("There is no matching contest");
    else
        System.out.println(contest.getContestName()+"\n"+contest.getPoints());;
}
public static int findTotalPointsBasedOnWinner(Contest[] contests, String contestWinner){
    int totalPoints = 0;
    for(Contest contest: contests){
        if(contest.getContestWinner().equalsIgnoreCase(contestWinner))
            totalPoints += contest.getPoints();
    }
    return totalPoints;
}
public static Contest getContestWithSecondHighestPoints(Contest[] contests, String category){
    List<Contest> newList = new ArrayList<Contest>();
    for(Contest contest: contests){
        if(contest.getCategory().equalsIgnoreCase(category))
            newList.add(contest);
    }
```

```
        if(newList.size()<=1) return null;
        Collections.sort(newList, new Sort());
        return newList.get(newList.size()-2);
    }
}
class Sort implements Comparator<Contest>{
    @Override
    public int compare(Contest obj1, Contest obj2) {
        return (int)(obj1.getPoints() - obj2.getPoints());
    }
}
class Contest{
private int contestId;
private String contestName;
private String contestWinner;
private int points;
private String category;
public Contest(int contestId, String contestName, String contestWinner, int points, String category){
    this.contestId = contestId;
    this.contestName = contestName;
    this.contestWinner = contestWinner;
    this.points = points;
    this.category = category;
}
public int getContestId() {
```

```
    return contestId;
}
public String getContestName() {
    return contestName;
}
public String getContestWinner(){
    return contestWinner;
}
public int getPoints() {
    return points;
}
public String getCategory() {
      return category;
   }
}
```

Q5-

```
import java.util.Scanner;
public class Solution
{

   public static void main(String[] args)
   {
      //code to read values
      int bottleId;
      String bottleBrand;
```

```
String bottleType;
int capacity;
String material;
double price;
Bottle[] bottles = new Bottle[4];
Scanner s = new Scanner(System.in);
for(int iterator = 0; iterator < bottles.length; iterator++){
   bottleId = s.nextInt();s.nextLine();
   bottleBrand = s.nextLine();
   bottleType = s.nextLine();
   capacity = s.nextInt();s.nextLine();
   material = s.nextLine();
   price = s.nextDouble();
   bottles[iterator] = new Bottle(bottleId, bottleBrand, bottleType, capacity, material, price);
}
s.nextLine();
String newMaterialValue = s.nextLine();
String newBottleBrandValue = s.nextLine();
s.close();
//code to call required method
int averagePriceBasedOnMaterial = getAvgPriceBasedOnMaterial(bottles, newMaterialValue);
Bottle secondHighestBottlePrice = getBottleBySecondHighestPrice(bottles, newBottleBrandValue);
//code to display the result
if(averagePriceBasedOnMaterial > 0)
   System.out.println(averagePriceBasedOnMaterial);
```

```
        else
            System.out.println("There is no matching bottles with the given material");

        if(secondHighestBottlePrice == null)
            System.out.println("Bottles are not available for the given brand");
        else
            System.out.println("Bottle Brand :"+secondHighestBottlePrice.getBottleBrand()+",Bottle Capacity
in ml :"+secondHighestBottlePrice.getCapacity()+",Price :"+secondHighestBottlePrice.getPrice());

    }

    //code the first method
    public static int getAvgPriceBasedOnMaterial(Bottle[] bottles, String material){
        double totalCartPrice = 0;
        int count = 0;
        for(int iterator = 0; iterator < bottles.length; iterator++){
            if(material.equalsIgnoreCase(bottles[iterator].getMaterial())){
                totalCartPrice += bottles[iterator].getPrice();
                count++;
            }
        }
        if(count > 0)
            return (int)totalCartPrice / count;
        else
            return 0;
    }
```

```
//code the second method
public static Bottle getBottleBySecondHighestPrice(Bottle[] bottles, String bottleBrand){
   int count = 0;
   for(int i = 0; i < bottles.length; i++){
      if(bottleBrand.equalsIgnoreCase(bottles[i].getBottleBrand()))
         count++;
   }
   if(count == 0) return null;
   Bottle[] b = new Bottle[count];
   count = 0;
   for(int i = 0; i < bottles.length; i++){
      if(bottleBrand.equalsIgnoreCase(bottles[i].getBottleBrand())){
         b[count++] = bottles[i];
      }
   }
   for(int i=0; i < b.length; i++){
      for(int j=0; j < b.length; j++){
         if(b[i].getPrice() < b[j].getPrice()){
            Bottle temp = b[i];
            b[i] = b[j];
            b[j] = temp;
         }
      }
   }
```

```
        return b[1];
    }

}

//code the class
class Bottle{
    private int bottleId;
    private String bottleBrand;
    private String bottleType;
    private int capacity;
    private String material;
    private double price;
    public Bottle(){}
    public Bottle(int bottleId, String bottleBrand, String bottleType, int capacity, String material, double
price){
        this.bottleId = bottleId;
        this.bottleBrand = bottleBrand;
        this.bottleType = bottleType;
        this.capacity = capacity;
        this.material = material;
        this.price = price;
    }
    public int getBottleId(){
        return bottleId;
    }
```

```
public String getBottleBrand(){
   return bottleBrand;
}
public String getBottleType(){
   return bottleType;
}
public int getCapacity(){
   return capacity;
}
public String getMaterial(){
   return material;
   }
   public double getPrice(){
      return price;
   }
}
```

Q-6

```
import java.util.*;
public class Solution{
  public static void main(String[] args) {
   Scanner s = new Scanner(System.in);
   int number = s.nextInt();
   Bill[] bill = new Bill[number];
   for(int i=0;i<bill.length;i++) {
    int billNo = s.nextInt();s.nextLine();
```

```
  String name = s.nextLine();
  String typeOfConnection = s.nextLine();
  double billAmount = s.nextDouble();s.nextLine();
  boolean status = s.nextBoolean();s.nextLine();
  bill[i] = new Bill(billNo, name, typeOfConnection, billAmount, status);
 }
 boolean parameter = s.nextBoolean();s.nextLine();
 String newTypeOfConnection = s.nextLine();
 s.close();

 Bill[] b = findBillWithMaxBillAmountBasedOnStatus(bill, parameter);
  if(b == null)
   System.out.println("There are no bill with the given status");
  else {
   for(int i=0;i<b.length;i++)
     System.out.println(b[i].getBillNo()+"#"+b[i].getName());

  }

  int countOfBills = getCountWithTypeOfConnection(bill, newTypeOfConnection);
  if(countOfBills == 0)
   System.out.println("There are no bills with given type of connection");
  else
   System.out.println(countOfBills);
 }
```

```
public static Bill[] findBillWithMaxBillAmountBasedOnStatus(Bill[] bill, boolean parameter) {
 int count = 0;
 for(int i=0;i<bill.length;i++) {
  if(bill[i].getStatus() == parameter)
    count++;
 }
 if(count == 0) return null;
 Bill[] b = new Bill[count];
 Bill[] anotherBill = new Bill[count];
 count = 0;
 for(int i=0;i<bill.length;i++) {
  if(bill[i].getStatus() == parameter) {
    b[count++] = bill[i];
  }
 }
 double maxBill = 0;
 for(int i=0;i<b.length;i++) {
  if(b[i].getBillAmount() > maxBill)
    maxBill = b[i].getBillAmount();
 }
 count = 0;
 for(int i=0;i<b.length;i++){
  if(b[i].getBillAmount() == maxBill)
    count++;
 }
```

```
Bill[] newBill = new Bill[count];
for(int i=0;i<b.length;i++) {
 for(int j=i+1;j<b.length;j++) {
  if(b[i].getBillNo()>=b[j].getBillNo()) {
   Bill temp = b[i];
   b[i] = b[j];
   b[j] = temp;
  }
 }
}
count = 0;
 for(int i=0;i<b.length;i++) {
  if(maxBill == b[i].getBillAmount())
   newBill[count++] = b[i];
 }
 return newBill;
}
public static int getCountWithTypeOfConnection(Bill[] bill, String newTypeOfConnection) {
 int count = 0;
 for(int i=0;i<bill.length;i++) {
  if(newTypeOfConnection.equalsIgnoreCase(bill[i].getTypeOfConnection()))
   count++;
 }
 return count;
}
```

```
}

class Bill{
 int billNo;
 String name;
 String typeOfConnection;
 double billAmount;
 boolean status;
 Bill(int billNo, String name, String typeOfConnection, double billAmount, boolean status){
  this.billNo = billNo;
  this.name = name;
  this.typeOfConnection = typeOfConnection;
  this.billAmount = billAmount;
  this.status = status;
 }
int getBillNo() {
 return billNo;
}
String getName() {
 return name;
}
String getTypeOfConnection() {
 return typeOfConnection;
}
double getBillAmount() {
```

```
    return billAmount;
  }
  boolean getStatus() {
    return status;
  }
}
```

Q7:-

```
import java.util.*;
public class Solution{
        public static void main(String[] args) {
                String songName;
                int duration;
                String artists;
                String genre;
                Scanner s = new Scanner(System.in);
                Album[] album = new Album[4];
                for(int i=0;i<album.length;i++) {
                        songName = s.nextLine();
                        duration = s.nextInt();s.nextLine();
                        artists = s.nextLine();
                        genre = s.nextLine();
                        album[i] = new Album(songName, duration, artists, genre);
                }
                String getGenre = s.nextLine();
                String getArtists = s.nextLine();
```

```
s.close();
String[] findSong = findSongBasedonGenre(album, getGenre);
Album[] a = findAlbumBasedonArtists(album, getArtists);

if(findSong == null)
        System.out.println("There are no songs with the given genre");
else
        for(int i=0;i<findSong.length;i++) {
                System.out.println(findSong[i]);
        }
                if(a==null)
                        System.out.println("There are no album with the given artists");
                else
                        for(int i=0;i<a.length;i++) {

        System.out.println(a[i].getSongName()+"#"+a[i].getDuration()+"#"+a[i].getArtists()+"#"+a[i].get
Genre());
                        }
        }
        public static String[] findSongBasedonGenre(Album[] album, String genre) {
                int count = 0;
                for(int i=0;i<album.length;i++) {
                        if(genre.equalsIgnoreCase(album[i].getGenre())) {
                                if(album[i].getDuration()>5)
                                        count++;
```

```
            }
}
if(count == 0) return null;
String[] song = new String[count];
count = 0;
for(int i=0;i<album.length;i++) {
            if(genre.equalsIgnoreCase(album[i].getGenre())) {
                        if(album[i].getDuration()>5)
                                    song[count++] = album[i].getSongName();
            }
}
            return so
}
public static Album[] findAlbumBasedonArtists(Album[] album, String artists) {
            int count = 0;
            for(int i=0;i<album.length;i++) {
                        if(album[i].getArtists().equalsIgnoreCase(artists)){
                                    if(album[i].getGenre().equalsIgnoreCase("Melody"))
                                                count++;
                        }
            }
            if(count == 0) return null;
            Album[] a = new Album[count];
            count = 0;
            for(int i=0;i<album.length;i++) {
```

```
            if(album[i].getArtists().equalsIgnoreCase(artists)){
                    if(album[i].getGenre().equalsIgnoreCase("Melody"))
                            a[count++] = album[i];
            }
}
for(int i=0;i<a.length;i++) {
            for(int j=i+1;j<a.length;j++) {
                    if(a[i].getSongName().compareTo(a[j].getSongName())>0) {
                            Album temp = a[i];
                            a[i] = a[j];
                            a[j] = temp;
                            }
                    }
            }
            return a;
    }
}

class Album{
        String songName;
        int duration;
        String artists;
        String genre;
        Album(String songName, int duration, String artists, String genre){
                this.songName = songName;
```

```
        this.duration = duration;
        this.artists = artists;
        this.genre = genre;
}
String getSongName() {
        return songName;
}
int getDuration() {
        return duration;
}
String getArtists() {
        return artists;
    }
    String getGenre() {
        return genre;
    }
}
```

Q8:-

```
import java.util.*;
public class Solution{
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int theatreId, seatCapacity, theatreRating;
        String theatreName;
        double ticketRate;
```

```
		Theatre[] th = new Theatre[4];
		for(int i=0;i<th.length;i++) {
			theatreId = s.nextInt();s.nextLine();
			theatreName = s.nextLine();
			seatCapacity = s.nextInt();
			ticketRate = s.nextDouble();
			theatreRating = s.nextInt();
			th[i] = new Theatre(theatreId, theatreName, seatCapacity, ticketRate,
theatreRating);
		}
		int getTheatreId = s.nextInt();
		s.close();
		String theatreCategory = findTheatreCategory(th, getTheatreId);
		Theatre t = findSecondHighestTicket(th);

		if(theatreCategory == null)
			System.out.println("There is no Theatre with the given theatreId");
		else
			System.out.println(theatreCategory);

		if(t == null)
			System.out.println("Only low rating theatres are available");
		else
			System.out.println(t.getTheatreName());
}
public static String findTheatreCategory(Theatre[] th, int theatreId) {
```

```
        String s=null;
        for(int i=0;i<th.length;i++) {
                if(theatreId == th[i].getTheatreId() ) {
                        if(th[i].getSeatCapacity()>1000 && th[i].getTheatreRating()>=4)
                                s = "Premium";
                        else
                                s = "Non Premium";

        }
        return s;
}
public static Theatre findSecondHighestTicket(Theatre[] th) {
        int count = 0;
        for(int i=0;i<th.length;i++) {
                if(th[i].getTheatreRating()>=2) count++;
        }
        if(count == 0) return null;
        Theatre[] t = new Theatre[count];
        count = 0;
        for(int i=0;i<th.length;i++) {
                if(th[i].getTheatreRating()>=2)
                        t[count++] = th[i];
        }
        for(int i=0;i<t.length;i++) {
                for(int j=i+1;j<t.length;j++) {
```

```
                        if(t[i].getTicketRate() <= t[j].getTicketRate()) {
                                Theatre temp = t[i];
                                t[i] = t[j];
                                t[j] = temp;
                        }
                    }
                }
                return t[1];
        }
}
class Theatre{
        int theatreId, seatCapacity, theatreRating;
        String theatreName;
        double ticketRate;
        Theatre(int theatreId, String theatreName, int seatCapacity, double ticketRate, int
theatreRating){
                this.theatreId = theatreId;
                this.theatreName = theatreName;
                this.seatCapacity = seatCapacity;
                this.ticketRate = ticketRate;
                this.theatreRating = theatreRating;
        }
        int getTheatreId() {
                return theatreId;
        }
        String getTheatreName() {
```

```
        return theatreName;
}
int getSeatCapacity() {
        return seatCapacity;
}
double getTicketRate() {
        return ticketRate;
}
int getTheatreRating() {
        return theatreRating;
}
}
```

Q9:-

```
import java.util.*;
public class Solution {
        public static void main(String[] args) {
                int soapId;
                String soapName;
                String brand;
                double price;
                boolean isGlycerin;
                Soap[] soap = new Soap[4];
                Scanner s = new Scanner(System.in);
                for(int i=0;i<soap.length;i++) {
                        soapId = s.nextInt();s.nextLine();
```

```
            soapName = s.nextLine();
            brand = s.nextLine();
            price = s.nextDouble();
            isGlycerin = s.nextBoolean();
            soap[i] = new Soap(soapId, soapName, brand, price, isGlycerin);
}
s.nextLine();
String getBrand = s.nextLine();
double minPrice = s.nextDouble();
double maxPrice = s.nextDouble();
Soap sp = getMaxPricedSoapByBrand(soap, getBrand);
            Soap[] ss = getGlycerinSoapsByPriceRange(soap, minPrice, maxPrice);

            if(sp == null)
                  System.out.println("There are no Soaps with given brand");
            else {
      System.out.println(sp.getSoapId()+"#"+sp.getSoapName()+"#"+sp.getBrand()+"#"+sp.getPrice());
            }
            for(int i=0;i<ss.length;i++) {
                  System.out.println(ss[i].getSoapId()+"-"+ss[i].getSoapName()+"-
"+ss[i].getPrice());
            }
      }
      public static Soap getMaxPricedSoapByBrand(Soap[] soapArr, String brand)
       {
```

```
//method logic
        double maxPrice = 0;
        Soap soap = new Soap();
        for(int i=0;i<soapArr.length;i++) {
                for(int j=0;j<soapArr.length;j++) {
                        if(soapArr[i].getPrice()<=soapArr[i].getPrice()) {
                                Soap temp = soapArr[i];
                                soapArr[i] = soapArr[j];
                                soapArr[j] = temp;
                        }
                }
                for(int i=0;i<soapArr.length;i++) {
                        if(brand.equalsIgnoreCase(soapArr[i].getBrand()))
                                return soapArr[i];
                }
                return null;
        }
        public static Soap[] getGlycerinSoapsByPriceRange(Soap[] soapArr, double minPrice, double
maxPrice)
        {
         //method logic
                int count = 0;
                for(int i=0;i<soapArr.length;i++) {
                        if(soapArr[i].getIsGlycerin()==true && soapArr[i].getPrice()>=min
soapArr[i].getPrice()<=maxPrice)
```

```
                    count++;
                }
                if(count == 0) return null;
                Soap[] soap = new Soap[count];
                count = 0;
                for(int i=0;i<soapArr.length;i++) {
                        if(soapArr[i].getIsGlycerin()==true && soapArr[i].getPrice()>=minPrice &&
soapArr[i].getPrice()<=maxPrice) {
                            soap[count++] = soapArr[i];
                        }
                }
                for(int i=0;i<soap.length;i++) {
                        for(int j=0;j<soap.length;j++) {
                            if(soap[i].getPrice()<=soap[i].getP
                                Soap temp = soap[i];
                                soap[i] = soap[j];
                                soap[j] = temp;
                            }
                        }
                }
                return soap;
            }
}

class Soap{
        int soapId;
```

```
String soapName;
String brand;
double price;
boolean isGlycerin;
Soap(){}
Soap(int soapId, String soapName, String brand, double price, boolean isGlycerin){
        this.soapId = soapId;
        this.soapName = soapName;
        this.brand = brand;
        this.price = price;
        this.isGlycerin = isGlycerin;
}
int getSoapId() {
        return soapId;
}
String getSoapName() {
        return soapName;
}
String getBrand() {
        return brand;
}
double getPrice() {
        return price;
}
boolean getIsGlycerin() {
```

```
                return isGlycerin;
        }
}
```

Q10:-

```
import java.util.*;


public class Solution{
        public static void main(String[] args) {
                int playerId;
                String playerName;
                String country;
                int goals;
                Scanner s = new Scanner(System.in);
                Football[] football = new Football[5];
                for(int i=0;i<football.length;i++) {
                        playerId = s.nextInt();s.nextLine();
                        playerName = s.nextLine();
                        country = s.nextLine();
                        goals = s.nextInt();
                        football[i] = new Football(playerId, playerName, country, goals);
                }
                s.nextLine();
                String getCountry1 = s.nextLine();
                String getCountry2 = s.nextLine();
                s.close();
```

```
int totalGoals = findGoalsForCountry(football, getCountry1);

Football[] fb = getGoalsAscendingOrder(football, getCountry2);


if(totalGoals == 0)

        System.out.println("No valid country specified");

else

        System.out.println(totalGoals);


if(fb == null)

        System.out.println("Country specified is wrong");

else {

                for(int i=0; i<fb.length; i++) {


        System.out.println(fb[i].getPlayerId()+"#"+fb[i].getPlayerName()+"#"+fb[i].getCountry()+"#"+fb[i
].getGoals());

                }

        }

}

public static int findGoalsForCountry(Football[] football, String country) {

        int sum = 0;

        for(int i=0; i<football.length; i++) {

                if(country.equalsIgnoreCase(football[i].getCo

                        sum += football[i].getGoals();

                }

        }

        return sum;
```

```
}
public static Football[] getGoalsAscendingOrder(Football[] football, String country) {
        int count = 0;
        for(int i=0; i<football.length; i++) {
                if(country.equalsIgnoreCase(football[i].getCountry())) {
                        count++;
                }
        }
        if(count == 0)
                return null;
        Football[] fb = new Football[count];
        count = 0;
        for(int i=0; i<football.length; i++) {
                if(country.equalsIgnoreCase(football[i].getCountry())) {
                        fb[count++] = football[i];
                }
        }
        for(int i=0; i<fb.length; i++) {
                for(int j=i+1; j<fb.length; j++) {
                        if(fb[i].getGoals() >= fb[i].getGoals()) {
                                Football temp = fb[i];
                                fb[i] = fb[j];
                                fb[j] = temp;
                        }
                }
```

```
                }
                return fb;
        }
}

class Football{
        int playerId;
        String playerName;
        String country;
        int goals;
        Football(int playerId, String playerName, String country, int goals) {
                this.playerId = playerId;
                this.playerName = playerName;
                this.country = country;
                this.goals = goals;
}
int getPlayerId() {
                return playerId;
}
String getPlayerName() {
                return playerName;
}
String getCountry() {
                return country;
}
```

```
        int getGoals() {
                return goals;
        }
}
```

Q11:-

```
import java.util.Scanner;

public class Main {

  public static void main(String[] args) {
// TODO Auto-generated method stub
Scanner sc=new Scanner(System.in);
Amazonprime[] ap=new Amazonprime[4];
for(int i=0;i<4;i++)
{
  int a=sc.nextInt();sc.nextLine();
  String b=sc.nextLine();
  int c=sc.nextInt();sc.nextLine();
  String d=sc.nextLine();
  int e=sc.nextInt();sc.nextLine();
  ap[i]=new Amazonprime(a,b,c,d,e);
}
int par1=sc.nextInt();sc.nextLine();
int par=sc.nextInt();sc.nextLine();
String show=sc.nextLine();
```

```
int remaindays=findRemainingSubcriptionDays(ap,par,par1);
if(remaindays!=0)
{
 System.out.println(remaindays);
}
else
 System.out.println("Its time to recharge your Prime Account");

Amazonprime amp[]=findDetailsForGivenShow(ap,show);
if(amp!=null)
{
  for(int i=0;i<amp.length;i++)
  {
     if(amp[i]!=null){
     System.out.println(amp[i].getPrimeId()+"$"+amp[i].getUserName()+"$"+amp[i].getViews());
  }
  }
}
else{
  System.out.println("No such shows available");
}

}
public static int findRemainingSubcriptionDays(Amazonprime[] ap,int par,int par1) {
 int remaindays=0;
```

```
for(int i=0;i<ap.length;i++)
{
 if(ap[i].getPrimeId()==par)
 {
 remaindays=((ap[i].getSubscribedPackage())-par1);
 }
}
return remaindays;
}

public static Amazonprime[] findDetailsForGivenShow(Amazonprime[] ap,String show) {
Amazonprime amp[]=new Amazonprime[4];
int j=0;
for(int i=0;i<ap.length;i++)
{
 if(ap[i].getShowStreaming().equalsIgnoreCase(show))
{
   amp[j++]=ap[i];
}
}
for(int i=0;i<j-1;i++)
{
 for(int k=0;k<j-1-i;k++)
{
   if(amp[k].getViews()>amp[k+1].getViews())
```

```
{
  Amazonprime aja=amp[k];
  amp[k]=amp[k+1];
  amp[k+1]=aja;
 }
}
 }
 if(j!=0)
  {
     return amp;
  }
     else{
        return null;
     }
  }
}
class Amazonprime
{
  int primeId;
  String userName;
  int subscribedPackage;
  String showStreaming;
  int views;
  public Amazonprime(int primeId, String userName, int subscribedPackage, String showStreaming, int
views) {
    this.primeId = primeId;
```

```
  this.userName = userName;
  this.subscribedPackage = subscribedPackage;
  this.showStreaming = showStreaming;
  this.views = views;
}
public int getPrimeId() {
  return primeId;
}
public void setPrimeId(int primeId) {
  this.primeId = primeId;
}
public String getUserName() {
  return userName;
}
public void setUserName(String userName) {
  this.userName = userName;
}
public int getSubscribedPackage() {
  return subscribedPackage;
}
public void setSubscribedPackage(int subscribedPackage) {
  this.subscribedPackage = subscribedPackage;
}
public String getShowStreaming() {
  return showStreaming;
```

```
}
public void setShowStreaming(String showStreaming) {
  this.showStreaming = showStreaming;
}
public int getViews() {
  return views;
}
public void setViews(int views) {
  this.views = views;
}
}
```

Q13:-

```
import java.util.*;
public class Solution{
   public static void main(String[] args){
      int playerId;
      String skill;
      String level;
      int points;
      Scanner s = new Scanner(System.in);
      Player[] player = new Player[4];
      for(int i=0; i<player.length; i++){
         playerId = s.nextInt();s.nextLine();
         skill = s.nextLine();
```

```
    level = s.nextLine();
    points = s.nextInt();
    player[i] = new Player(playerId, skill, level, points);
}
s.nextLine();
String getSkill = s.nextLine();
String getLevel = s.nextLine();
s.close();
int getPoints = findPointsForGivenSkill(player, getSkill);
Player p = getPlayerBasedOnLevel(player, getSkill, getLevel);

    if(getPoints == 0)
        System.out.println("The given Skill is not available");
    else
        System.out.println(getPoints);


    if(p == null)
        System.out.println("No player is available with specified level, skill and eligibility points");
    else
        System.out.println(p.getPlayerId());
}
public static int findPointsForGivenSkill(Player[] players, String skill) {
        int count = 0;
        for(int i=0; i<players.length; i++) {
                if(skill.equalsIgnoreCase(players[i].getSkill()))
```

```
                count += players[i].getPoints();
        }
        return count;
    }
    public static Player getPlayerBasedOnLevel(Player[] players, String skill, String level) {
        for(int i=0; i<players.length; i++) {
                if(skill.equalsIgnoreCase(players[i].getSkill()) &&
level.equalsIgnoreCase(players[i].getLevel()) && players[i].getPoints()>=20)
                return players[i];
        }
        return null;
    }
}
class Player{
        int playerId;
    String skill;
    String level;
    int points;
    Player(){}
    Player(int playerId, String skill, String level, int points){
        this.playerId = playerId;
        this.skill = skill;
        this.level = level;
        this.points = points;
    }
    int getPlayerId() {
```

```
        return playerId;
}
String getSkill() {
        return skill;
}
String getLevel() {
        return level;
}
int getPoints() {
        return points;
}
```

Q14:-

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution{
   public static void main(Stri
      int contentId;
      String content;
      String category;
      int likes;
```

```
int dislikes;
int shares;
Content[] c = new Content[4];
Scanner s = new Scanner(System.in);
for(int i=0; i<c.length; i++){
   contentId = s.nextInt();s.nextLine();
   content = s.nextLine();
   category = s.nextLine();
   likes = s.nextInt();
   dislikes = s.nextInt();
   shares = s.nextInt();
   c[i] = new Content(contentId, content, category, likes, dislikes, shares);
}
s.nextLine();
String requiredCategory = s.nextLine();
s.close();
Content newContent = getMaxSharedContent(c);
String newString = findContentByCategory(c, requiredCategory);

if(newContent == null)
   System.out.println("No content Shared");
else
   System.out.println("Maximum shared content:"+newContent.getContent());

if(newString == null)
```

```
        System.out.println("Content not available for the category");
    else
        System.out.println("Content Details:"+newString);
}
public static Content getMaxSharedContent(Content[] contentArray){
    Content c = null;
    int maxShare = 0;
    for(int i=0;i<contentArray.length;i++){
        if(contentArray[i].getShares() > maxShare){
            maxShare = contentArray[i].getShares();
            c = contentArray[i];
        }
    }
    return c;
  }
  public static String findContentByCategory(Content[] contentArray, String category){
    String newString;
    for(int i=0; i<contentArray.length; i++){
        if(category.equalsIgnoreCase(contentArray[i].getCatego
0){
            newString =
contentArray[i].getContentId()+"#"+contentArray[i].getContent()+"#"+contentArray[i].getCategory();
            return newString;
        }
    }
    return null;
```

```
    }
}
class Content{
    int contentId;
    String content;
    String category;
    int likes;
    int dislikes;
    int shares;
    Content(){}
    Content(int contentId, String content, String category, int likes, int dislikes, int shares){
        this.contentId = contentId;
        this.content = content;
        this.category = category;
        this.likes = likes;
        this.dislikes = dislikes;
        this.shares = shares;
}
int getContentId(){
    return contentId;
}
String getContent(){
    return content;
}
String getCategory(){
```

```
    return category;
}
int getLikes(){
    return likes;
}
int getDislikes(){
    return dislikes;
}
int getShares(){
    return shares;
}
}
```

Q15:-

```
import java.util.*;
public class MyClass {
    public static void main(String[] args){
        String songName;
        int duration;
        String artists, language;
        double rating;
        Scanner s = new Scanner(System.in);
        Songs[] songs = new Songs[5];
        for(int i=0; i<songs.length; i++){
            songName = s.nextLine();
            duration = s.nextInt();s.nextLine();
```

```
    artists = s.nextLine();
    language = s.nextLine();
    rating = s.nextDouble();s.nextLine();
    songs[i] = new Songs(songName, duration, artists, language, rating);
}
String newLanguage = s.nextLine();
String newArtist = s.nextLine();
s.close();
String[] newSongs = findSongBasedOnLanguage(songs, newLanguage);
if(newSongs == null)
    System.out.println("There are no songs with the given language");
    else
      for(String i: newSongs)
          System.out.println(i);
    Songs[] songs1 = findSongBasedOnArtists(songs, newArtist);
    if(songs1 == null)
      System.out.println("There are no songs with the given artists");
    else
      for(Songs song: songs1)
          System.out.println(song.getSongName()+"\n"+song.getRating());
}
public static String[] findSongBasedOnLanguage(Songs[] songs, String language){
    List<String> newList = new ArrayList<>();
    for(Songs song : songs){
      if(song.getLanguage().equalsIgnoreCase(language))
```

```
            newList.add(song.getSongName());
        }
        if(newList.size() == 0) return null;
        Collections.sort(newList, Comparator.reverseOrder());
        String[] arr = newList.toArray(new String[newList.size()]);
        return arr;
    }
    public static Songs[] findSongBasedOnArtists(Songs[] songs, String artist){
        List<Songs> newList = new ArrayList<>();
        for(Songs song : songs){
            if(song.getArtists().equalsIgnoreCase(artist))
                newList.add(song);
            }
            if(newList.size() == 0) return null;
            Collections.sort(newList, new Sort());
            Songs[] arr = newList.toArray(new Songs[newList.size()]);
            return arr;
        }
    }
    class Sort implements Comparator<Songs>{
        public int compare(Songs obj1, Songs obj2){
            return (int)(obj1.getRating() - obj2.getRating());
        }
    }
    class Songs{
```

```
private String songName;
private int duration;
private String artists;
private String language;
private double rating;
public Songs(String songName, int duration, String artists, String language, double rating){
    this.songName = songName;
    this.duration = duration;
    this.artists = artists;
    this.language = language;
    this.rating = rating;
}
public String getSongName() {
    return songName;
}
public int getDuration() {
    return duration;
}
public String getArtists() {
    return artists;
}
public String getLanguage() {
    return language;
}
public double getRating() {
```

```
        return rating;
    }
}
```

Q16:-

```
import java.util.*;
import java.lang.*;
public class MyClass {
    public static void main(String[] args){
        Scanner s = new Scanner(System.in);
        Theatre[] theatres = new Theatre[4];
        for(int i=0; i< theatres.length; i++){
            int theatreId = s.nextInt();s.nextLine();
            String theatreName = s.nextLine();
            int seatCapacity = s.nextInt();s.nextLine();
            double ticketRate = s.nextDouble();
            double theatreRating = s.nextDouble();
            boolean balconyAvailable = s.nextBoolean();
            theatres[i] = new Theatre(theatreId, theatreName, seatCapacity, ticketRate, theatreRating,
balconyAvailable);
        }
        int newTheatreId = s.nextInt();
        int newCapacity = s.nextInt();
        s.close();
        String theatreCategory = findTheatreCategory(theatres, newTheatreId);
        if(theatreCategory == null)
            System.out.println("There is no Theatre with the given Theatre ID");
```

```
    else
        System.out.println(theatreCategory);

    Theatre[] newTheatres = searchTheatreByCategory(theatres, newCapacity);
    if(newTheatres == null)
        System.out.println("Only  low capacity theatres available");
    else
        for(Theatre theatre: newTheatres){
            System.out.println(theatre.getTheatreId());
        }
}
  public static String findTheatreCategory(Theatre[] theatres, int theatreId){
      for(Theatre theatre: theatres){
          if(theatre.getTheatreId() == theatreId) {
              if (theatre.isBalconyAvailable() && theatre.getTheatreRating() > 4)
                  return "Ultra Premium";
              else if (theatre.isBalconyAvailable() && (theatre.getTheatreRating() >= 3 &&
theatre.getTheatreRating() < 4))
                  return "Premium";
              else
                  return "Normal";
          }
      }
      return null;
  }
  public static Theatre[] searchTheatreByCategory(Theatre[] theatres, int seatCapacity){
```

```
    List<Theatre> newList = new ArrayList<Theatre>();
    for(Theatre theatre: theatres){
        if(theatre.getSeatCapacity() > seatCapacity)
            newList.add(theatre);
    }
    if(newList.size() == 0) return null;
    Collections.sort(newList, new Sort());
    Theatre[] newTheatres = newList.toArray(new Theatre[newList.size()]);
    return newTheatres;
  }
}
class Sort implements Comparator<Theatre>{
    public int compare(Theatre obj1, Theatre obj2){
        return (int)(obj1.getTicketRate() - obj2.getTicketRate());
    }
}
class Theatre{
    private int theatreId;
    private String theatreName;
    private int seatCapacity;
    private double ticketRate;
    private double theatreRating;
    private boolean balconyAvailable;
    public Theatre(int theatreId, String theatreName, int seatCapacity, double ticketRate, double
theatreRating, boolean balconyAvailable){
        this.theatreId = theatreId;
```

```
    this.theatreName = theatreName;
    this.seatCapacity = seatCapacity;
    this.ticketRate = ticketRate;
    this.theatreRating = theatreRating;
    this.balconyAvailable = balconyAvailable;
}
public int getTheatreId() {
    return theatreId;
}
public String getTheatreName() {
    return theatreName;
    }
    public int getSeatCapacity() {
        return seatCapacity;
    }
    public double getTicketRate() {
        return ticketRate;
    }
    public double getTheatreRating() {
        return theatreRating;
    }
    public boolean isBalconyAvailable() {
        return balconyAvailable;
    }
}
```

Q17:-

```
import java.util.*;
import java.lang.*;
public class MyClass {
   public static void main(String[] args){
      Scanner s = new Scanner(System.in);
      Employee[] employees = new Employee[4];
      for(int i=0; i<employees.length; i++){
         int employeeId = s.nextInt();s.nextLine();
         String name = s.nextLine();
         String branch = s.nextLine();
         double rating = s.nextDouble();
         boolean companyTransport = s.nextBoolean();
         employees[i] = new Employee(employeeId, name, branch, rating, companyTransport);
      }
      s.nextLine();
      String newBranch = s.nextLine();
      s.close();
      int count = findCountOfEmployeesUsingCompTransport(employees, newBranch);
      if(count > 0)
         System.out.println(count);
      else
         System.out.println("No such Employees");

      Employee employee = findEmployeeWithSecondHighestRating(employees);
```

```
    if(employee == null)
        System.out.println("All Employees using company transport");
    else
        System.out.println(employee.getEmployeeId()+"\n"+employee.getName());
}
public static int findCountOfEmployeesUsingCompTransport(Employee[] employees, String branch){
    int count = 0;
    for(Employee employee: employees){
        if(employee.getBranch().equalsIgnoreCase(branch) && employee.isCompanyTransport())
            count++;
        }
        return count;
    }
    public static Employee findEmployeeWithSecondHighestRating(Employee[] employees){
        List<Employee> newList = new ArrayList<Employee>();
        for(Employee employee: employees){
            if(!employee.isCompanyTransport())
                newList.add(employee);
        }
        if(newList.size()<2) return null;
        Collections.sort(newList, new Sort());
        return newList.get(newList.size()-2);
    }
}
class Sort implements Comparator<Employee>{
```

```
    public int compare(Employee obj1, Employee obj2){
        return (int)(obj1.getRating() - obj2.getRating());
    }
}
class Employee{
    private int employeeId;
    private String name, branch;
    private double rating;
    private boolean companyTransport;
    public Employee(int employeeId, String name, String branch, double rating, boolean
companyTransport){
        this.employeeId = employeeId;
        this.name = name;
        this.branch = branch;
        this.rating = rating;
        this.companyTransport = companyTransport;
}
public int getEmployeeId() {
    return employeeId;
}
public String getName(){
    return name;
}
public String getBranch(){
    return branch;
}
```

```
    public double getRating() {
        return rating;
    }
    public boolean isCompanyTransport() {
        return companyTransport;
    }
}
```

Q18:-

```
import java.util.*;
public class MyClass {
public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    Doctor d[]=new Doctor[4];
    for(int i=0;i<d.length;i++) {
        int id=in.nextInt();in.nextLine();
        String name = in.nextLine();
        double fee = in.nextDouble();in.nextLine();
        boolean avail=in.nextBoolean();in.nextLine();
        String dept = in.nextLine();
        d[i] = new Doctor(id,name,fee,avail,dept);
    }
    String dp = in.nextLine();
    double totalFee = findTotalFeeByDepartment(d, dp);
    if(totalFee == 0) {
        System.out.println("Department not found");
```

```
}
else {
   System.out.println(totalFee);
}
ArrayList<Doctor> list = searchDoctorByAvailability(d);
if(list == null) {
   System.out.println("No doctor available on sunday");
}
else {
   list.forEach( id -> System.out.println(id.getId()));
   }
}
public static double findTotalFeeByDepartment(Doctor d[],String dept) {
   double total=0;
   for(Doctor i : d) {
      if(i.getDept().equalsIgnoreCase(dept)) {
         total += i.getFee();
      }
   }
   if(total == 0) {
      return 0;
   }
   else {
      return total;
   }
```

```
}
public static ArrayList<Doctor> searchDoctorByAvailability(Doctor d[]) {
   ArrayList<Doctor> list = new ArrayList<Doctor>();
   for(Doctor  i : d) {
      if(i.isAvailable()) {
         list.add(i);
      }
   }
   if(list.size() == 0) {
      return null;
      }
      else {
         Comparator<Doctor> mylist = Comparator.comparing(Doctor :: getId);
         Collections.sort(list,mylist);
         return list;
      }
   }
}
class Doctor{
   private int id;
   private String name;
   private double fee;
   boolean available;
   private String dept;
   public Doctor(int id, String name, double fee, boolean available, String dept) {
```

```
    this.id = id;
    this.name = name;
    this.fee = fee;
    this.available = available;
    this.dept = dept;
}
public int getId() {
    return id;
}
public String getName() {
    return name;
    }
    public double getFee() {
        return fee;
    }
    public boolean isAvailable() {
        return available;
    }
    public String getDept() {
        return dept;
    }
}
```

Q19:-

```
import java.util.*;
class MyClass
```

```
{
   public static void main(String args[])
   {
      Scanner sc=new Scanner(System.in);
      int invoicenumber;
      String fromcity;
      String tocity;
      String orderdate;
      String deliverydate;
      double price;
Package p[]=new Package[5];
for(int i=0;i<p.length;i++)
{
   invoicenumber=sc.nextInt();
   sc.nextLine();
   fromcity=sc.nextLine();
   tocity=sc.nextLine();
   orderdate=sc.nextLine();
   deliverydate=sc.nextLine();
   price=sc.nextDouble();
   p[i]=new Package(invoicenumber,fromcity,tocity,orderdate,deliverydate,price);
}
sc.nextLine();
String c1=sc.nextLine();
String c2=sc.nextLine();
```

```
sc.close();
int count=countOrdersDeliveredInAWeek(p,c1);
if(count!=0)
   System.out.println(count);
else
   System.out.println("No package delivered");
Package pck=searchPackageByCity(p,c2);
if(pck!=null)
{
   System.out.println(pck.getinvoice());
   System.out.println(pck.getprice());
   }
   else
      System.out.println("No package found");
}
public static int countOrdersDeliveredInAWeek(Package p[], String c1)
{
   int count=0;
   for(Package i:p)
   {
      if(c1.equalsIgnoreCase(i.getfromcity()))
      {

         String od=i.getorderdate().substring(0,2);
         String dd=i.getdeliverydate().substring(0,2);
```

```
            int od1=Integer.parseInt(od);
            int dd1=Integer.parseInt(dd);
            if(dd1-od1<7)
               count++;
        }
    }
    return count;
}
public static Package searchPackageByCity(Package p[], String c2)
{
ArrayList<Double> ar=new ArrayList<>();
Package pck=new Package();

for(Package i:p)
{
   if(c2.equalsIgnoreCase(i.gettocity()))
   {
      ar.add(i.getprice());
   }
}
if(ar.size()!=0)
{
   Collections.sort(ar,Collections.reverseOrder());
   double pr=ar.get(1);
   for(Package i:p)
```

```
        {
            if(pr==i.getprice())
            {
                pck=i;
                return pck;
            }
        }
    }
    return null;
}
}
class Package
{
    private int invoicenumber;
    private String fromcity,tocity,orderdate,deliverydate;
    private double price;
    Package()
    {
    }
    Package(int invoicenumber, String fromcity, String tocity, String orderdate, String deliverydate, double
price)
    {
        this.invoicenumber=invoicenumber;
        this.fromcity=fromcity;
        this.tocity=tocity;
        this.orderdate=orderdate;
```

```
    this.deliverydate=deliverydate;
    this.price=price;
}
public int getinvoice()
{
    return invoicenumber;
}
public String getfromcity()
{
    return fromcity;
}
public String gettocity()
{
    return tocity;
}
public String getorderdate()
{
    return orderdate;
}
public String getdeliverydate()
{
    return deliverydate;
}
public double getprice()
{
```

```
        return price;
    }
}
import java.util.*;
import java.lang.*;
public class MyClass{
    public static void main(String[] args){
        int guestId;
        String guestName;
        String dateOfBooking;
        int noOfRoomsBooked;
        String mealOption;
        double totalBill;
        Scanner s = new Scanner(System.in);
        ResortGuest[] resortGuests = new ResortGuest[4];
        for(int i=0; i<resortGuests.length; i++){
            guestId = s.nextInt();s.nextLine();
            guestName = s.nextLine();
            dateOfBooking = s.nextLine();
            noOfRoomsBooked = s.nextInt();s.nextLine();
            mealOption = s.nextLine();
            totalBill = s.nextDouble();
            resortGuests[i] = new ResortGuest(guestId, guestName, dateOfBooking, noOfRoomsBooked,
mealOption, totalBill);
        }
        s.nextLine();
```

```
String month = s.nextLine();
String mealOpted = s.nextLine();
s.close();

int roomsBooked = findNumberOfRoomsBookedInMonth(resortGuests, month);
if(roomsBooked == 0)
   System.out.println("No rooms booked");
else
   System.out.println(roomsBooked);

   ResortGuest resortGuest = searchResortGuestByMealOpted(resortGuests, mealOpted);
   if(resortGuest == null)
      System.out.println("No such meal");
   else
      System.out.println(resortGuest.getGuestId());
}
public static int findNumberOfRoomsBookedInMonth(ResortGuest[] resortGuests, String month){
   int countOfRoomsBooked = 0;
   for(ResortGuest resortGuest: resortGuests){
      String tempMonth = resortGuest.getDateOfBooking().substring(3,6);
      if(tempMonth.equalsIgnoreCase(month))
         countOfRoomsBooked++;
   }
   return countOfRoomsBooked;
}
```

```
  public static ResortGuest searchResortGuestByMealOpted(ResortGuest[] resortGuests, String
mealOpted){
    List<ResortGuest> tempList = new ArrayList<ResortGuest>();
    for(ResortGuest resortGuest: resortGuests){
      if(resortGuest.getMealOption().equalsIgnoreCase(mealOpted)){
        tempList.add(resortGuest);
      }
    }
    if(tempList.size() == 0) return null;
    Collections.sort(tempList, new Comparison());
      return tempList.get(tempList.size()-2);
    }
}
class Comparison implements Comparator<ResortGuest>{
  public int compare(ResortGuest obj1, ResortGuest obj2){
    return (int)(obj1.getTotalBill() - obj2.getTotalBill());
  }
}
class ResortGuest{
  private int guestId;
  private String guestName;
  private String dateOfBooking;
  private int noOfRoomsBooked;
  private String mealOption;
  private double totalBill;
```

```
  public ResortGuest(int guestId, String guestName, String dateOfBooking, int noOfRoomsBooked,
String mealOption, double totalBill){
      this.guestId = guestId;
      this.guestName = guestName;
      this.dateOfBooking = dateOfBooking;
      this.noOfRoomsBooked = noOfRoomsBooked;
      this.mealOption = mealOption;
      this.totalBill = totalBill;
  }
  public int getGuestId(){
    return guestId;
  }
  public String getGuestName(){
    return guestName;
  }
  public String getDateOfBooking(){
    return dateOfBooking;
  }
  public int getNoOfRoomsBooked(){
    return noOfRoomsBooked;
  }
  public String getMealOption(){
    return mealOption;
  }
  public double getTotalBill(){
    return totalBill;
```

```
    }
}
```

Q20:-

```
import java.util.*;
import java.lang.*;

public class MyClass{
    public static void main(String[] args){
        int voterId;
        String voterName;
        int voterAge;
        boolean isVoteCasted;
        String constituency;
        Scanner scanner = new Scanner(System.in);
        Vote[] votes = new Vote[4];
        for(int i=0; i<votes.length; i++){
            voterId = scanner.nextInt();scanner.nextLine();
            voterName = scanner.nextLine();
            voterAge = scanner.nextInt();
            isVoteCasted = scanner.nextBoolean();scanner.nextLine();
            constituency = scanner.nextLine();
            votes[i] = new Vote(voterId, voterName, voterAge, isVoteCasted, constituency);
        }
        String checkConstituency = scanner.nextLine();
        scanner.close();
```

```
int totalVotesCasted = findTotalVotesCastedByConstituency(votes, checkConstituency);
if(totalVotesCasted == 0)
   System.out.println("No votes casted");
else
   System.out.println(totalVotesCasted);
int[] voterAgeArray = searchVoterByAge(votes);
if(voterAgeArray == null)
   System.out.println("No such voters");
else
   for(int voter: voterAgeArray)
      System.out.println(voter);
}
public static int findTotalVotesCastedByConstituency(Vote[] votes, String constituency){
   int totalVotesCasted = 0;
   for(Vote vote: votes){
      if(constituency.equalsIgnoreCase(vote.getConstituency())){
         if(vote.getIsVoteCasted())
            totalVotesCasted++;
      }
   }
   return totalVotesCasted;
}
public static int[] searchVoterByAge(Vote[] votes){
   ArrayList<Integer> arrayOfVoterIdList = new ArrayLis          >();
   for(Vote vote: votes){
```

```
            if(vote.getVoterAge() < 30){
                arrayOfVoterIdList.add(vote.getVoterId());
            }
        }
        if(arrayOfVoterIdList.size() == 0) return null;
        Collections.sort(arrayOfVoterIdList);
        int[] array = arrayOfVoterIdList.stream().mapToInt(i -> i).toArray();
        return array;
    }
}

class Vote{
    private int voterId;
    private String voterName;
    private int voterAge;
    private boolean isVoteCasted;
    private String constituency;
    public Vote(int voterId, String voterName, int voterAge, boolean isVoteCasted, String constituency){
        this.voterId = voterId;
        this.voterName = voterName;
        this.voterAge = voterAge;
        this.isVoteCasted = isVoteCasted;
        this.constituency = constituency;
    }
    public int getVoterId(){
```

```
    return voterId;
}
public String getVoterName(){
    return voterName;
}
public int getVoterAge(){
    return voterAge;
}
public boolean getIsVoteCasted(){
    return isVoteCasted;
    }
    public String getConstituency(){
        return constituency;
    }
}
```

Q21:-

```
import java.util.*;


public class MyClass{
    public static void main(String[] args){
        int applianceId;
        String applianceName;
        String applianceCategory;
        double applianceAmount;
        boolean insurance;
```

```
        Appliance[] appliances = new Appliance[4];
        Scanner s = new Scanner(System.in);
        for(int i=0;i<appliances.length;i++){
            applianceId = s.nextInt();s.nextLine();
            applianceName = s.nextLine();
            applianceCategory = s.nextLine();
            applianceAmount = s.nextDouble();
            appliances[i] = new Appliance(applianceId, applianceName, applianceCategory,
applianceAmount);
        }
        s.nextLine();
        int applianceId1 = s.nextInt();
        boolean insurance1 = s.nextBoolean();s.nextLine();
        String applianceCategory1 = s.nextLine();

        double applianceAmount1 = getApplianceAmount(appliances, applianceId1, insurance1);
        System.out.println(applianceAmount1);

        Appliance appl = getCostliestAppliance(appliances, applianceCategory1);
        System.out.println("Appliance Id: "+appl.getApplianceId()+" Appliance Name:
"+appl.getApplianceName()+" Total Amount: "+appl.getApplianceAmount());
    }

    public static double getApplianceAmount(Appliance[] appliances, int applianceId, boolean insurance){
        double newApplianceAmount = 0;
```

```
    for(int i=0;i<appliances.length;i++){
      if(appliances[i].getApplianceId() == applianceId){
        if(insurance == true){
          newApplianceAmount = appliances[i].getApplianceAmount() +
((appliances[i].getApplianceAmount() * 20)/100);
          appliances[i].setApplianceAmount(newApplianceAmount);
        }
      }
    }
    return newApplianceAmount;
  }

  public static Appliance getCostliestAppliance(Appliance[] appliances, String applianceCategory){
    double maxCost = -999;
    Appliance appliances1 = null;
    for(int i=0;i<appliances.length;i++){
      if(applianceCategory.equalsIgnoreCase(appliances[i].getApplianceCategory())){
        if(appliances[i].getApplianceAmount() > maxCost){
          maxCost = appliances[i].getApplianceAmount();
          appliances1 = appliances[i];
        }
      }
    }
    return appliances1;
  }
}
```

```
class Appliance{
   private int applianceId;
   private String applianceName;
   private String applianceCategory;
   private double applianceAmount;
   public Appliance(int applianceId, String applianceName, String applianceCategory, double
applianceAmount){
      this.applianceId = applianceId;
      this.applianceName = applianceName;
      this.applianceCategory = applianceCategory;
      this.applianceAmount = applianceAmount;
}
public int getApplianceId(){
   return applianceId;
}
public String getApplianceCategory(){
   return applianceCategory;
}
public String getApplianceName(){
   return applianceName;
}
public void setApplianceAmount(double applianceAmount){
   this.applianceAmount = applianceAmount;
}
public double getApplianceAmount(){
```

```
        return applianceAmount;
    }
}
```

Q22:-

```
import java.util.Scanner; //importing scanner to read data from keyboard
public class Solution{ //this program is executed in Onlinegdb, so class name is Main
    public static void main(String[] args){
        Scanner s =new Scanner(System.in); //creating scanner object for reading data

        int regNo;
        String agencyName;
        String packageType;
        int price;
        boolean flightFacility; //variable names given in question

        TravelAgencies[] ta =new TravelAgencies[4]; //creating array of objects

        for(int i=0;i<4;i++){ //reading values from keyboard for each variable
            regNo =s.nextInt();s.nextLine();
            agencyName =s.nextLine();
            packageType =s.nextLine();
            price =s.nextInt();
            flightFacility =s.nextBoolean();
            ta[i] =new TravelAgencies(regNo, agencyName, packageType, price, flightFacility); //assigning
values for each object
        }
```

```
        int getRegNo = s.nextInt();s.nextLine(); //constraint 1 from question
        String getPackageType = s.nextLine(); //constraint 2 from question
        s.close(); //closing scanner object

        int highestPackagePrice = findAgencyWithHighestPackagePrice(ta);
        TravelAgencies travelAgencies = agencyDetailsForGivenIdAndType(ta, getRegNo, getPackageType);
//calling static methods from main function

        System.out.println(highestPackagePrice); //printing highest price
        if(travelAgencies == null)
            System.out.println("A string value should be printed here!");
        else
            System.out.println(travelAgencies.getAgencyName()+"\n"+travelAgencies.getPrice());
            //printing agencyName and price with required constraints from question
    }
    public static int findAgencyWithHighestPackagePrice(TravelAgencies[] agencies){
        int maxPrice = agencies[0].getPrice(); //taking object1 price value as max
        for(int i=1;i<agencies.length;i++){
            if(agencies[i].getPrice() > maxPrice)
                maxPrice = agencies[i].getPrice(); //comparing all prices to get maximum
        }
        return maxPrice; // returning maximum price from all objects
    }
    public static TravelAgencies agencyDetailsForGivenIdAndType(TravelAgencies[] agencies, int regNo,
String packageType){
```

```
    TravelAgencies ta = new TravelAgencies(); //creating an object
    for(int i=0;i<agencies.length;i++){
      if(agencies[i].getFlightFacility()){ //condition 1 checking
        if(agencies[i].getRegNo()==regNo &&
packageType.equalsIgnoreCase(agencies[i].getPackageType())){ //condition 2 checking
          ta = agencies[i]; //if 2 conditions satisfy, assigning object
          return ta; //returning required object to main function
        }
      }
    }
    return null; //if no conditions satisfy, null value is returned
  }
}

class TravelAgencies{
  int regNo;
  String agencyName;
  String packageType;
  int price;
  boolean flightFacility;
  TravelAgencies(){} //empty constuctor
  TravelAgencies(int regNo, String agencyName, String packageType, int price, boolean flightFacility){
//parameterized constructor
    super();
    this.regNo = regNo;
    this.agencyName = agencyName;
```

```
    this.packageType = packageType;
    this.price = price;
    this.flightFacility = flightFacility;
}
int getRegNo(){ //getter for regNo
    return regNo;
}
String getAgencyName(){ //getter for agencyName
    return agencyName;
}
    String getPackageType(){ //getter for packageType
        return packageType;
    }
    int getPrice(){ //getter for price
        return price;
    }
    boolean getFlightFacility(){ //getter for flightFacility
        return flightFacility;
    }
}
```

Q23:-

```
import java.util.Scanner;

class Item{
    int itemId;
    String itemName;
```

```
String itemType;
double itemPrice;
Item(int itemId, String itemName, String itemType, double itemPrice){
   this.itemId = itemId;
   this.itemName = itemName;
   this.itemType = itemType;
   this.itemPrice = itemPrice;
}
int getItemId(){
   return itemId;
   }
   String getItemName(){
      return itemName;
   }
   String getItemType(){
      return itemType;
   }
   double getItemPrice(){
      return itemPrice;
   }
}

public class Solution1{
   public static void main(String[] args){
      int itemId;
```

```
String itemName;

String itemType;

double itemPrice;

Item[] item = new Item[4];

Scanner s = new Scanner(System.in);

for(int i=0;i<item.length;i++){

   itemId = s.nextInt();s.nextLine();

   itemName = s.nextLine();

   itemType = s.nextLine();

   itemPrice = s.nextDouble();

   item[i] = new Item(itemId, itemName, itemType, itemPrice);

}

s.nextLine();

String newItemType = s.nextLine();

String newItemName = s.nextLine();

s.close();


int avgItemPrice = findAvgItemPriceByTypes(item, newItemType);

Item[] items = searchItemByName(item, newItemName);


if(avgItemPrice > 0)

   System.out.println(avgItemPrice);

else

   System.out.println("There are no
```

```
    if(items == null)
        System.out.println("There are no items with the given name");
    else{
        for(int i=0;i<items.length;i++){
            System.out.println(items[i].getItemId());
        }
    }
}
public static int findAvgItemPriceByTypes(Item[] item, String itemType){
    int avgPrice = 0, count = 0;
    double sum = 0;
    for(int i=0;i<item.length;i++){
        if(itemType.equalsIgnoreCase(item[i].getItemType())) {
            sum += item[i].getItemPrice();
            count++;
        }
    }
    avgPrice = (int)(sum/count);
    return avgPrice;
}
public static Item[] searchItemByName(Item[] item, String itemName){
    int count = 0;
    for(int i=0;i<item.length;i++){
        if(itemName.equalsIgnoreCase(item[i].getItemName()))
            count++;
```

```
}
if(count == 0) return null;
Item[] items = new Item[count];
count = 0;
for(int i=0;i<item.length;i++){
   if(itemName.equalsIgnoreCase(item[i].getItemName())){
      items[count++] = item[i];
   }
}
for(int i=0;i<items.length;i++){
      for(int j=i+1;j<items.length;j++){
         if(items[i].getItemId() >= items[j].getItemId()){
            Item temp = items[i];
            items[i] = items[j];
            items[j] = temp;
         }
      }
   }
   return items;
  }
}
```

Q24:-

```
import java.io.*;
import java.util.*;
import java.text.*;
```

```
import java.math.*;
import java.util.regex.*;

public class Solution
{

 public static void main(String[] args)
 {
  //code to read values
  //code to call required method
 //code to display the result
 int songId;
 String title;
 String artist;
 int rating;
 Scanner s = new Scanner(System.in);
 Song[] song = new Song[5];
 for(int i=0;i<5;i++){
    songId = s.nextInt();s.nextLine();
    title = s.nextLine();
    artist = s.nextLine();
    rating = s.nextInt();
    song[i] = new Song(songId, title, artist, rating);
 }
 s.nextLine();
```

```
String artist1 = s.nextLine();

String artist2 = s.nextLine();

s.close();


int findAvg = findAvgRatingForArtist(song, artist1);

Song[] newSong = searchSongByArtist(artist2, song);


if(findAvg == 0)

  System.out.println("There are no songs with the given artist");

else

     System.out.println(findAvg);


 if(newSong == null)

   System.out.println("There are no songs available for the given artist");

 else{

    for(int i=0;i<newSong.length;i++){

        System.out.println(newSong[i].getSongId());

    }

 }


}


public static int findAvgRatingForArtist(Song[] song, String artist)

 {

  //method logic
```

```
int avgRating = 0, count = 0;
for(int i=0;i<song.length;i++){
   if(artist.equalsIgnoreCase(song[i].getArtist())){
      avgRating += song[i].getRating();
      count++;
   }
}
if(avgRating > 0) return avgRating/count;
else return avgRating;
}
```

```
public static Song[] searchSongByArtist(String artist, Song[] song)
 {
 //method logic
 int count = 0;
 for(int i=0;i<song.length;i++){
    if(artist.equalsIgnoreCase(song[i].getArtist()))
     count++;
 }
 if(count == 0)
  return null;
 Song[] s = new Song[count];
 count = 0;
 for(int i=0;i<song.length;i++){
    if(artist.equalsIgnoreCase(song[i].getArtist())){
```

```
      s[count++] = song[i];
    }
  }
  for(int i=0;i<s.length;i++){
    for(int j=i+1;j<s.length;j++){
      if(s[i].getSongId()<=s[j].getSongId()){
        Song temp = s[i];
        s[i] = s[j];
        s[j] = temp;
      }
    }
  }
  return s;
  }
}

class Song
{
  //code to build the class
  int songId;
  String title;
  String artist;
  int rating;
  Song(int songId, String title, String artist, int rating){
    this.songId = songId;
```

```
    this.title = title;
    this.artist = artist;
    this.rating = rating;
}
int getSongId(){
    return songId;
}
String getTitle(){
    return title;
}
 String getArtist(){
    return artist;
 }
 int getRating(){
    return rating;
 }
}
```

Q25:-

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution
```

```
{


public static void main(String[] args)
{
 //code to read values
 int playerId;
 String playerName;
 int score1, score2, score3;
 Player[] players = new Player[4];
 Scanner s = new Scanner(System.in);
for(int i=0; i<4;i++){
   playerId = s.nextInt();s.nextLine();
   playerName = s.nextLine();
   score1 = s.nextInt();
   score2 = s.nextInt();
   score3 = s.nextInt();
   players[i] = new Player(playerId, player                                re3);
}
//code to call required method
int maxScore = findTotalHundredsCount(players);
Player player = getTopPlayer(players);
//code to display the result
if(maxScore != 0)
  System.out.println(maxScore);
else
```

```
  System.out.println("No Hundreds Scored in Tournament");
 System.out.println(player.getPlayerId()+"#"+player.getPlayerName());
}

public static int findTotalHundredsCount(Player[] players)
{
 //method logic
 int count = 0;
 for(int i=0; i<players.length; i++){
    if(players[i].getScore1()>=100)count++;
    if(players[i].getScore2()>=100)count++;
    if(players[i].getScore3()>=100)count++;
  }
 return count;
}

public static Player getTopPlayer(Player[] players)
{
 //method logic
 Player player = new Player();
 int maxScore = players[0].getScore1()+players[0].getScore2()+players[0].getScore3();
 player = players[0];
 for(int i=1;i<players.length;i++){
    int newScore = players[i].getScore1()+players[i].getScore2()+players[i].getScore3();
    if(newScore>=maxScore){
```

```
        maxScore = newScore;
        player = players[i];
      }
    }
    return player;
  }
}

class Player
{
  //code to build the class
  int playerId;
  String playerName;
  int score1, score2, score3;
  Player(){}
  Player(int playerId, String playerName, int score1, int score2, int score3){
     this.playerId = playerId;
     this.playerName = playerName;
     this.score1 = score1;
     this.score2 = score2;
     this.score3 = score3;
  }
  int getPlayerId(){
     return playerId;
  }
```

```
String getPlayerName(){
   return playerName;
}
int getScore1(){
   return score1;
}
int getScore2(){
   return score2;
}
 int getScore3(){
    return score3;
 }
}
```

Q26:-

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Main{
   public static void main(String[] args){
      //Creating variables
      String iMEICode; //iMEICode is of String type
      boolean isSingleSIM; //isSingleSIM is of Boolean type
```

```
String processor; //processor is of String type

double price; //price is of double type

String manufacturer; //manufacturer is of String type


//creating a scanner 's' object to read values from the keyboard and the syntax is shown below

Scanner s = new Scanner(System.in);


//creating array of Mobile Class objects by using the below statement

Mobile[] mobile = new Mobile[5];


//reading each variable for each object using a for loop

for(int i=0; i<5; i++){

   //reading iMEICode from keyboard using nextLine()

   iMEICode = s.nextLine();

   //reading isSingleSIM using nextBoolean()

   isSingleSIM = s.nextBoolean();s.nextLine();

   //here we need to use one more s.nextLine() statement inorder to read string correctly

   //reading processor using nextLine()

   processor = s.nextLine();

   // reading price using nextDouble()

   price = s.nextDouble();s.nextLine();

   //reading manufacturer using nextLine()

   manufacturer = s.nextLine();

   //assigning all variables to each object 'm'

   mobile[i] = new Mobile(iMEICode, isSingleSIM, processor, price, manufacturer);
```

```
}
//reading discountPercentage
double discountPercentage = s.nextDouble();s.nextLine();
//reading manufacturerName
String manufacturerName = s.nextLine();

//Calling function getCountOfValidIMEIMobiles
int countOfValidIMEIMobiles = getCountOfValidIMEIMobiles(mobile);
//Calling function findMobileWithMaxPrice
  Mobile result = findMobileWithMaxPrice(mobile, discountPercentage, manufacturerName);

  //Printing results on the screen using System.out.println()
  System.out.println(countOfValidIMEIMobiles);
  if(result == null) //checking whether result object is null or not
    System.out.println("No Mobile Found");
  else
    System.out.println(result.getIMEICode()+"@"+result.getPrice());
}

public static int getCountOfValidIMEIMobiles(Mobile[] listOfMobiles){
  //count variable to count number of Mobiles which have 15 digit IMEI code
  int count = 0;
  for(int i=0; i<listOfMobiles.length; i++){
    //checking IMEI code length and Single SIM status
    if(listOfMobiles[i].getIMEICode().length() == 15 && listOfMobiles[i].getIsSingleSIM() == true)
```

```
            count++; //If above condition true, then count increments
        }
        return count; //returning count of valid sims to main method
    }

    public static Mobile findMobileWithMaxPrice(Mobile[] listOfMobiles, double discountPercentage,
String manufacturerName){
        //creating a mobile object
        Mobile mobile = new Mobile();
        //Creating newPrice variable to get newPrice after applying discountPercentage
        double newPrice;
        for(int i=0;i<listOfMobiles.length;i++){
            //checking manufacturerName variable with manufacturer in all objects by ignoring case
            if(manufacturerName.equalsIgnoreCase(listOfMobiles[i].getManufacturer())){
                //formula for newPrice from the question
                newPrice = listOfMobiles[i].getPrice() - ((listOfMobiles[i].getPrice()*discountPercentage)/100);
                //setting newPrice for the mobile object which matches with manufacturer
                listOfMobiles[i].setPrice(newPrice);
                //assigning that object to 'mobile' object
                mobile = listOfMobiles[i];
                return mobile; //returning mobile object to main function
            }
        }
        return null;
    }
}
```

}

class Mobile{

String iMEICode;

boolean isSingleSIM;

String processor;

double price;

String manufacturer;

//creating an empty constructor because we are creating an object in findMobileWithMaxPrice function

Mobile(){}

//creating parameterized constructor

Mobile(String iMEICode, boolean isSingleSIM, String processor, double price, String manufacturer){

//if we have same variable names, we should use this keyword to store valules

this.iMEICode = iMEICode;

this.isSingleSIM = isSingleSIM;

this.processor = processor;

this.price = price;

this.manufacturer = manufacturer;

}

//creating getters and setters

String getIMEICode(){

return iMEICode;

}

boolean getIsSingleSIM(){

return isSingleSIM;

```
}
```

```
String getProcessor(){
   return processor;
}
void setPrice(double price){ //we need a setter to assign new price to object
   this.price = price;
}
double getPrice(){
   return price;
}
   String getManufacturer(){
      return manufacturer;
   }
}
```

Q27:-

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution
{

 public static void main(String[] args)
 {
```

```
//code to read values
Scanner s = new Scanner(System.in);
int vesselId, noOfVoyagesPlanned, noOfVoyagesCompleted;
String vesselName, purpose;
NavalVessel[] nv = new NavalVessel[4]; //creating array of objects
for(int i=0;i<nv.length;i++){
   vesselId = s.nextInt();s.nextLine();
   vesselName = s.nextLine();
   noOfVoyagesPlanned = s.nextInt();
   noOfVoyagesCompleted = s.nextInt();s.nextLine();
   purpose = s.nextLine();
   nv[i] = new NavalVessel(vesselId, vesselName, noOfVoyagesPlanned, noOfVoyagesCompleted,
purpose); //assigning the values to each object
 }
 int getPercentageValue = s.nextInt();s.nextLine();
 String getPurposeValue = s.nextLine();
 //code to call required method
 int avgOfVoyagesCompleted = findAvgVoyagesByPct(nv, getPercentageValue);

 if(avgOfVoyagesCompleted>0)
  System.out.println(avgOfVoyagesCompleted);
  else
     System.out.println("There are no voyages completed with this percentage");
 //code to display the result
```

```
  NavalVessel navalvessel = findVesselByGrade(nv, getPurposeValue);

  if(navalvessel == null)
     System.out.println("No Naval Vessel is available with the specified purpose");
  else
     System.out.println(navalvessel.getVesselName()+"%"+navalvessel.getClassification());
}

public static int findAvgVoyagesByPct(NavalVessel[] nvArray, int percentage)
{
  //method logic
  int avg = 0,count=0;
  for(int i=0;i<nvArray.length;i++){
     int percent = (nvArray[i].getNoOfVoyagesCompleted()*100)/nvArray[i].getNoOfVoyagesPlanned();
     if(percent >= percentage){
        avg += nvArray[i].getNoOfVoyagesCompleted();
        count++;
     }
  }
  if(avg == 0)
     return 0;
   else
     return avg/count;
 }

public static NavalVessel findVesselByGrade(NavalVessel[] nvArray, String purpose)
{
```

```
  //method logic
  NavalVessel nv = new NavalVessel();
  for(int i=0;i<nvArray.length;i++){
     if(purpose.equalsIgnoreCase(nvArray[i].getPurpose())){
        int percentage =
(nvArray[i].getNoOfVoyagesCompleted()*100)/nvArray[i].getNoOfVoyagesPlanned(); //finding
percentage
        if(percentage==100) nvArray[i].setClassification("Star");
        else if(percentage >=80 && percentage <=99) nvArray[i].setClassification("Leader");
        else if(percentage >=55 && percentage <=79) nvArray[i].setClassification("Inspirer");
        else
         nvArray[i].setClassification("Striver");
         nv = nvArray[i];
         return nv;
     }
  }
  return null;
 }
}

class NavalVessel
{
 //code to build the class
 int vesselId, noOfVoyagesPlanned, noOfVoyagesCompleted;
 String vesselName, purpose, classification;
 NavalVessel(){}
```

```
 NavalVessel(int vesselId, String vesselName, int noOfVoyagesPlanned, int noOfVoyagesCompleted,
String purpose){
    super();
    this.vesselId = vesselId;
    this.vesselName = vesselName;
    this.noOfVoyagesPlanned = noOfVoyagesPlanned;
    this.noOfVoyagesCompleted = noOfVoyagesCompleted;
    this.purpose = purpose;
  }
  int getVesselId(){
    return vesselId;
  }
  String getVesselName(){
    return vesselName;
  }
  int getNoOfVoyagesPlanned(){
    return noOfVoyagesPlanned;
  }
  int getNoOfVoyagesCompleted(){
    return noOfVoyagesCompleted;
  }
  String getPurpose(){
    return purpose;
  }
  void setClassification(String classification){
```

```
        this.classification = classification;
    }
    String getClassification(){
        return classification;
    }
}
```

Q28:-

```
import java.util.*;
class Main{
    public static void main(String[] args){
        Scanner s = new Scanner(System.in);
        int movieId, rating, budget;
        String director;
        Movie[] movies = new Movie[4]; //creating array of movie objects
        for(int i=0;i<4;i++){
            movieId = s.nextInt();s.nextLine();
            director = s.nextLine();
            rating = s.nextInt();
            budget = s.nextInt();
            movies[i] = new Movie(movieId, director, rating, budget);
        }
        s.nextLine();
        String findDirector = s.nextLine();
        int findRating = s.nextInt();
        int findBudget = s.nextInt();
        int findAvg = findAvgBudgetByDirector(movies, findDirector); //calling method
```

```
if(findAvg > 0){
   System.out.println(findAvg);
}
else
   System.out.println("Sorry - The given director has not directed any movie yet");
Movie m = getMovieByRatingBudget(movies, findRating, findBudget); //calling method
if(m == null)
   System.out.println("Sorry - No movie available at specified rating and budget requirement");
else
      System.out.println(m.getMovieId());
  }
  public static int findAvgBudgetByDirector(Movie[] movies, String director){
    int sum = 0,count=0;
    for(int i=0;i<4;i++){
      if(director.equalsIgnoreCase(movies[i].getDirector())){ //if required director matches with existing
directors in the array of objects
        count++;
        sum += movies[i].getBudget(); //adding the movie budgets
      }
    }
    if(sum>0)
      return sum/count; //returing the average movie budget
    else
      return 0;
  }
```

```
public static Movie getMovieByRatingBudget(Movie[] movies, int rating, int budget){

    Movie m = new Movie();

    for(int i=0;i<4;i++){

        if(movies[i].getRating()==rating && movies[i].getBudget()==budget){ //if rating and budget
matches, then we need to return that object to main method.

            if(movies[i].getBudget()%movies[i].getRating() == 0){

                m = movies[i];

                return m;

            }

        }

    }

        return null;

    }

}


class Movie{

    int movieId,rating,budget;

    String director;

    Movie(){}

    Movie(int movieId, String director, int rating, int budget){

        super();

        this.movieId = movieId;

        this.director = director;

        this.rating = rating;

        this.budget = budget;

    }
```

```
int getMovieId(){

    return movieId;
}
String getDirector(){
    return director;
}
int getRating(){
    return rating;
}
int getBudget(){
        return budget;
    }
}
```